第二章 递归与分治

Chapter 2 Recursion and Divided-conquer

计算机学院 高翔
gaoxg@nwpu.edu.cn

目 录

2.1 递归的概念

2.2 递归算法设计

2.3 递归算法示例

2.4 分治法的设计思想

2.5 快速排序

2.6 主方法

2.7归并排序

2.8 寻找最大次大元素

2.9 折半查找

2.10 寻找第k小元素

2.11 寻找两个有序序列的中位数

2.12 寻找最大子序列和

2.13 棋盘覆盖问题

2.1 递归的概念

递归的定义

在定义一个过程或函数时出现调用本过程或本函数的成分，称之为递归。若调用自身，称之为**直接递归**。若过程或函数p调用过程或函数q，而q又调用p，称之为**间接递归**。

任何间接递归都可以等价地转换为直接递归。

如果一个递归过程或递归函数中递归调用语句是最后一条执行语句，则称这种递归调用为**尾递归**。

【例2.1】设计求n!（n为正整数）的递归算法。

解：对应的递归函数如下：

```
int fun(int n)
{  if (n==1)                          //语句1
        return(1);                    //语句2
    else                          //语句3
        return(fun(n-1)*n);       //语句4
}
```

在该函数fun(n)求解过程中，直接调用fun(n-1)（语句4）自身，所以它是一个直接递归函数。又由于递归调用是最后一条语句，所以它又属于尾递归。

一般来说，能够用递归解决的问题应该满足以下三个条件：

- 需要解决的问题可以转化为一个或多个子问题来求解，而这些子问题的求解方法与原问题完全相同，只是在数量规模上不同。
- 递归调用的次数必须是有限的。
- 必须有结束递归的条件来终止递归。

何时使用递归

在以下三种情况下，常常要用到递归的方法。

1. 定义是递归的

有许多数学公式、数列等的定义是递归的。例如，求n!和Fibonacci数列等。这些问题的求解过程可以将其递归定义直接转化为对应的递归算法。

Recursive definition

Factorial function

$$n! = \begin{cases} 1, & \text{if } n = 0 \\ n * (n-1)!, & \text{if } n \geq 1 \end{cases}$$

Fibonacci numbers

$$Fib\ (n) = \begin{cases} n, & n = 0,1 \\ Fib\ (n-1) + Fib\ (n-2), & n > 1 \end{cases}$$

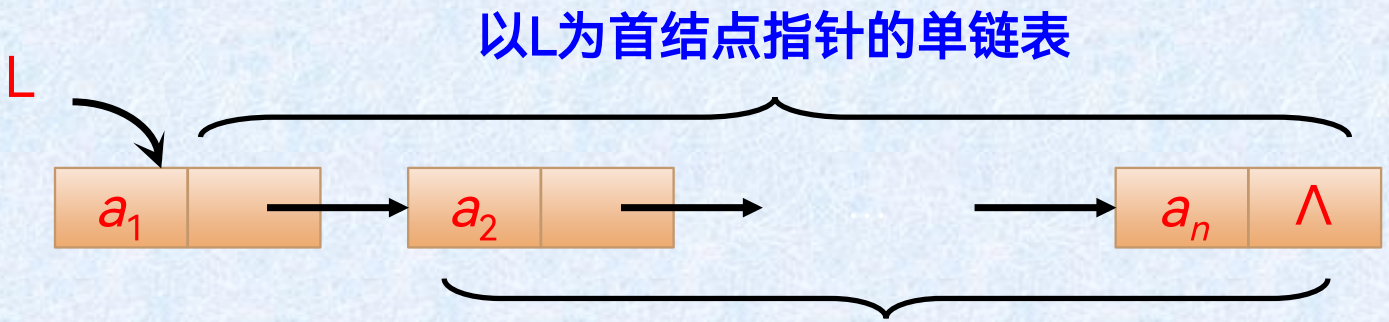
2.数据结构是递归的

有些数据结构是递归的。例如单链表就是一种递归数据结构，其结点类型声明如下：

```
typedef struct LNode
{   ElemType data;
    struct LNode *next;
} LinkList;
```

结构体LNode的定义中用到了它自身，即指针域next是一种指向自身类型的指针，所以它是一种递归数据结构。

不带头结点单链表示意图



体现出数据结构的递归性。

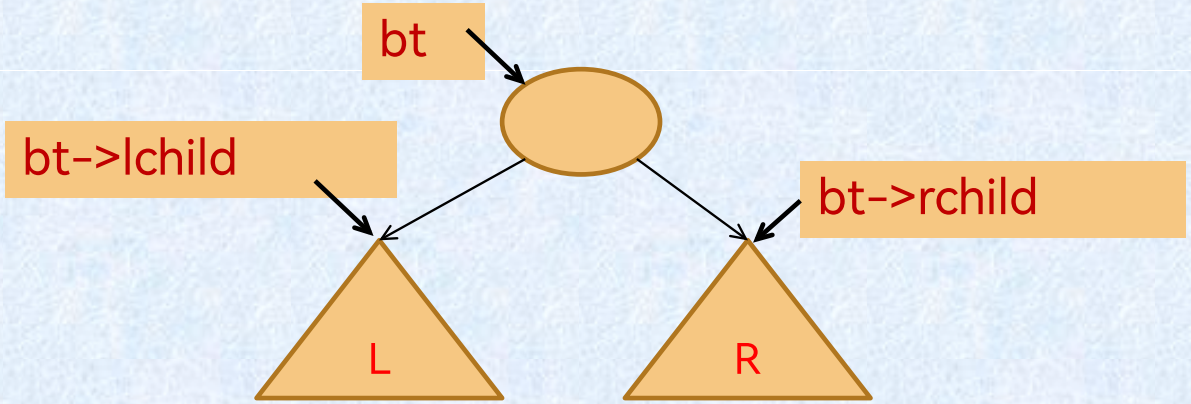
对于递归数据结构，采用递归的方法编写算法既方便又有效。例如，求一个不带头结点的单链表L的所有data域（假设为int型）之和的递归算法如下：

```
int Sum(LinkList *L)
{   if (L==NULL)
            return 0;
      else
            return(L->data+Sum(L->next));
}
```

【例2.2】分析二叉树的二叉链存储结构的递归性，设计求非空二叉链bt中所有结点值之和的递归算法，假设二叉链的data域为int型。

解：二叉树采用二叉链存储结构，其结点类型定义如下：

```
typedef struct BNode
{   int data;
    struct BNode *lchild，*rchild;
} BTNode;                    //二叉链结点类型
```



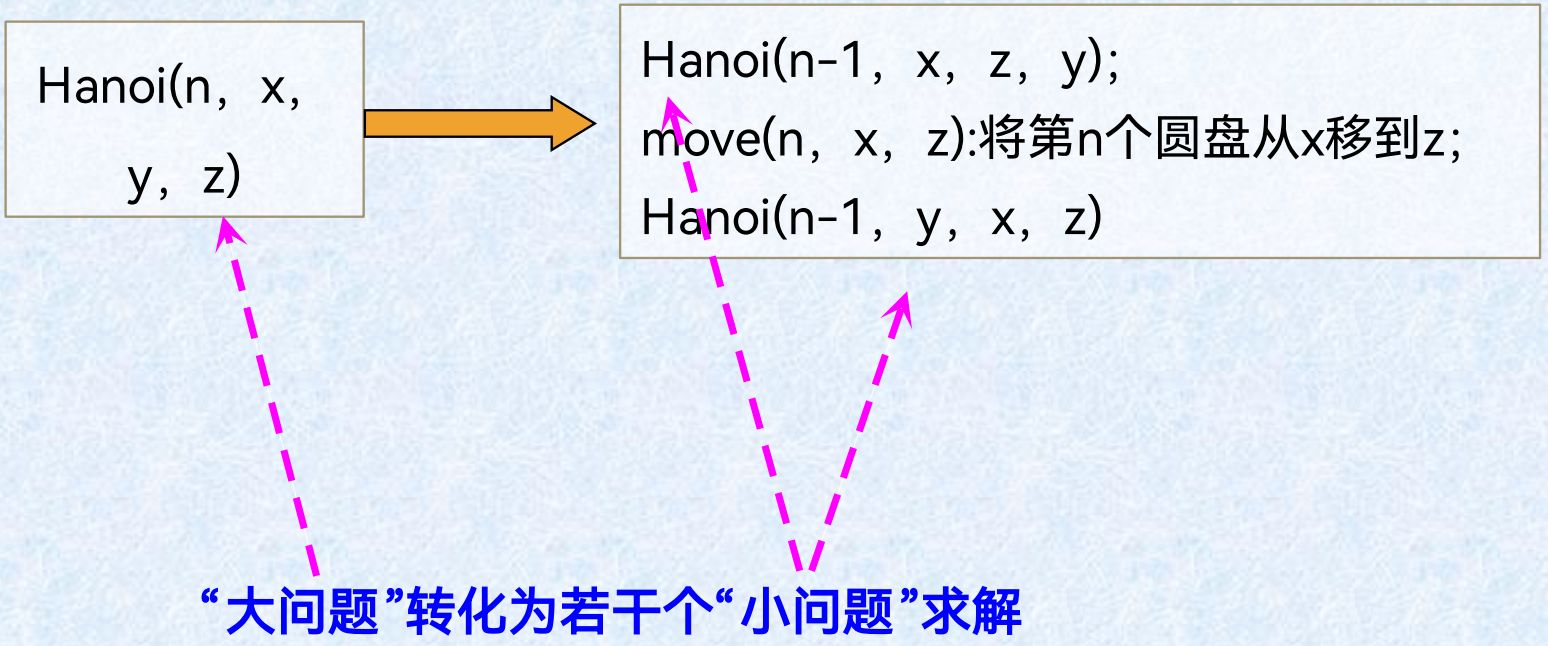

```
int Sumbt(BTNode *bt)          //求二叉树bt中所有结点值之和
{  if (bt->lchild==NULL && bt->rchild==NULL)
      return bt->data;   //只有一个结点时返回该结点值
   else   //否则返回左、右子树结点值之和加上根结点值
      return Sumbt(bt->lchild)+ Sumbt(bt->rchild)+
bt->data;
}
```

3.问题的求解方法是递归的

有些问题的解法是递归的，典型的有Hanoi问题求解。

盘片移动时必须遵守以下规则：每次只能移动一个盘片；盘片可以插在X、Y和Z中任一塔座；任何时候都不能将一个较大的盘片放在较小的盘片上。

设Hanoi(n，x，y，z)表示将n个盘片从x通过y移动到z上，递归分解的过程是:



递归模型

递归模型是递归算法的抽象，它反映一个递归问题的递归结构。例如前面的递归算法对应的递归模型如下：

fun(1)=1　　(1)

fun(n)=n*fun(n-1)　　n>1　(2)

其中，第一个式子给出了递归的终止条件，第二个式子给出了fun(n)的值与fun(n-1)的值之间的关系，我们把第一个式子称为递归出口，把第二个式子称为递归体。

一般地，一个递归模型是由递归出口和递归体两部分组成，前者确定递归到何时结束，后者确定递归求解时的递推关系。

递归出口的一般格式如下：

$$f(s_1)=m_1 \qquad (2.1)$$

这里的s1与m1均为常量，有些递归问题可能有几个递归出口。

递归体的一般格式如下：

$$f(s_{n+1})=g(f(s_i),\ f(s_{i+1}),\ \ldots,\ f(s_n),\ c_j,\ c_{j+1},\ \ldots,\ c_m)$$

其中，n、i、j和m均为正整数。这里的Sn+1是一个递归“大问题”，Si、Si+1、…、Sn为递归“小问题”，Cj、Cj+1、…、Cm是若干个可以直接（用非递归方法）解决的问题，g是一个非递归函数，可以直接求值。

递归算法的执行过程

- 一个正确的递归程序虽然每次调用的是相同的子程序，但它的参量、输入数据等均有变化。
- 在正常的情况下，随着调用的不断深入，必定会出现调用到某一层的函数时，不再执行递归调用而终止函数的执行，遇到递归出口便是这种情况。

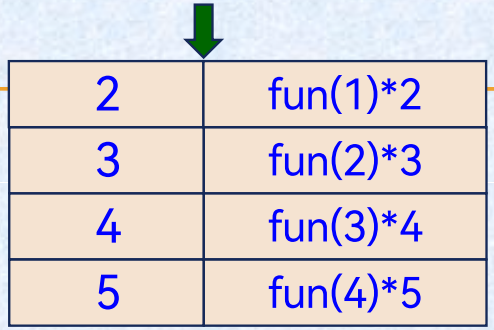
- 递归调用是函数嵌套调用的一种特殊情况，即它是调用自身代码。也可以把每一次递归调用理解成调用自身代码的一个复制件。
- 由于每次调用时，它的参量和局部变量均不相同，因而也就保证了各个复制件执行时的独立性。

- 系统为每一次调用开辟一组存储单元，用来存放本次调用的返回地址以及被中断的函数的参量值。
- 这些单元以系统栈的形式存放，每调用一次进栈一次，当返回时执行出栈操作，把当前栈顶保留的值送回相应的参量中进行恢复，并按栈顶中的返回地址，从断点继续执行。

对于例2.1的递归算法，求5!即执行fun(5)时内部栈的变化及求解过程如下：

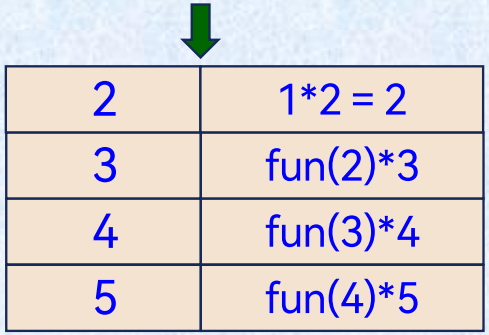
```
void main()
{  printf("%d\n",  fun(5));  }
```

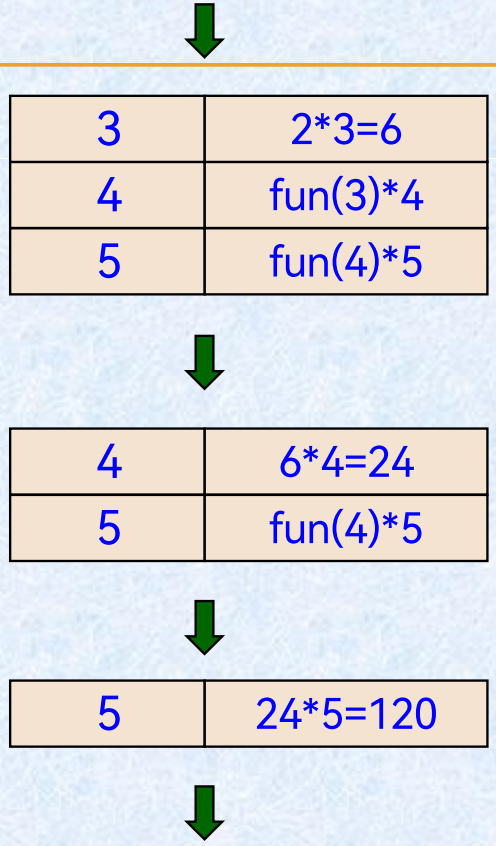
fun(2)调用：进栈



fun(1)调用：进栈并求值

退栈1次并求fun(2)值





退栈1次并求fun(3)值

退栈1次并求fun(4)值

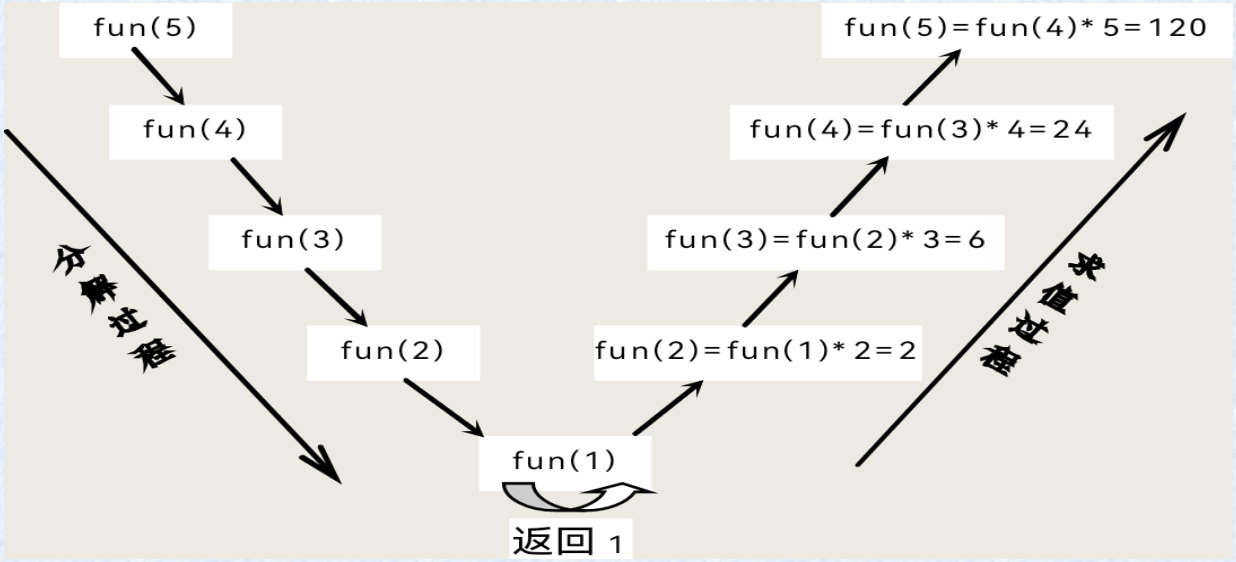
退栈1次并求fun(5)值

退栈1次并输出120

从以上过程可以得出：

- 每递归调用一次，就需进栈一次，最多的进栈元素个数称为递归深度，当n越大，递归深度越深，开辟的栈空间也越大。
- 每当遇到递归出口或完成本次执行时，需退栈一次，并恢复参量值，当全部执行完毕时，栈应为空。

归纳起来，递归调用的实现是分两步进行的，第一步是分解过程，即用递归体将“大问题”分解成“小问题”，直到递归出口为止，然后进行第二步的求值过程，即已知“小问题”，计算“大问题”。前面的fun(5)求解过程如下所示。



【例2.3】 Fibonacci数列定义为：

Fib(n)=1　　//n=1

Fib(n)=1　　//n=2

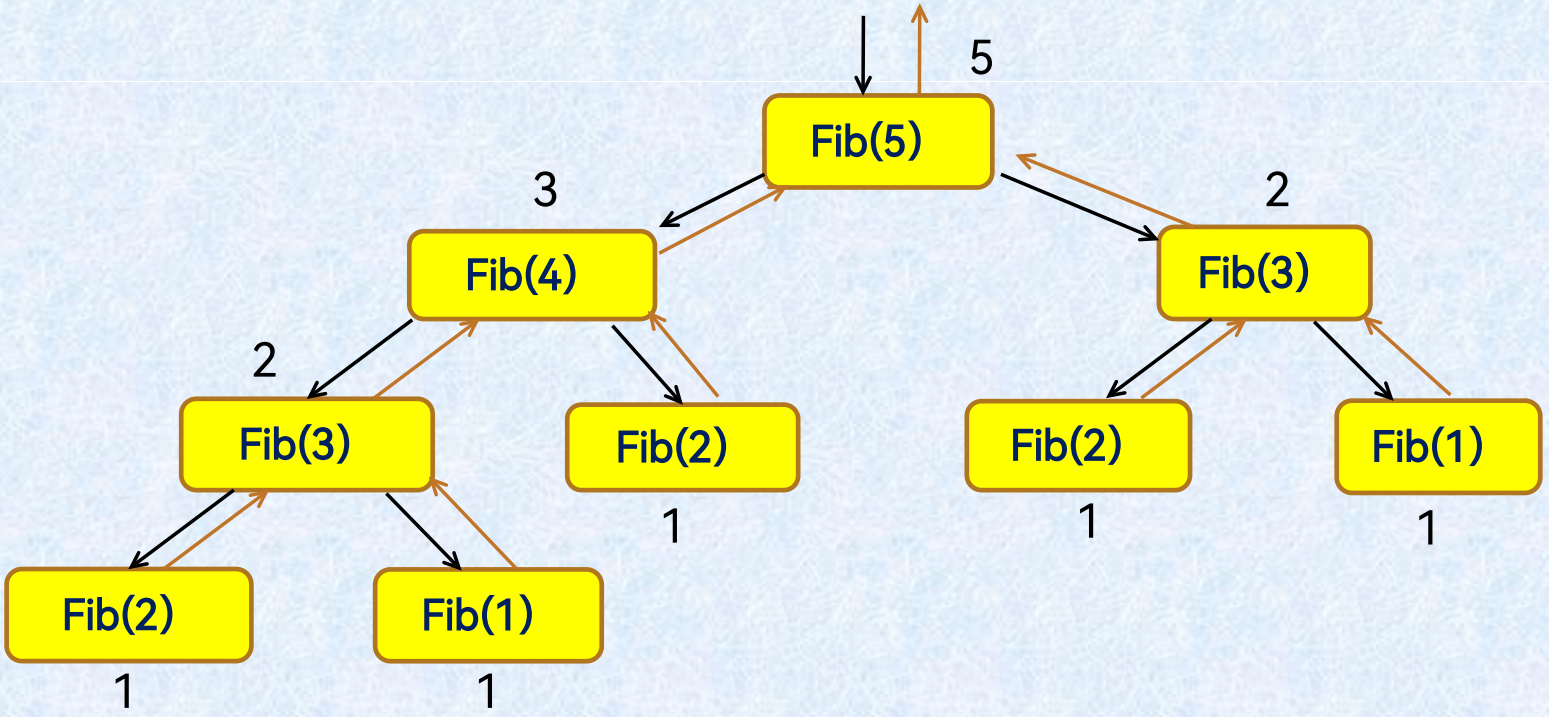
Fib(n)=Fib(n-1)+Fib(n-2)　　//n>2

对应的递归算法如下：

```
int Fib(int n)
{    if (n==1 || n==2)
            return 1;
      else
          return Fib(n-1)+Fib(n-2);
}
```

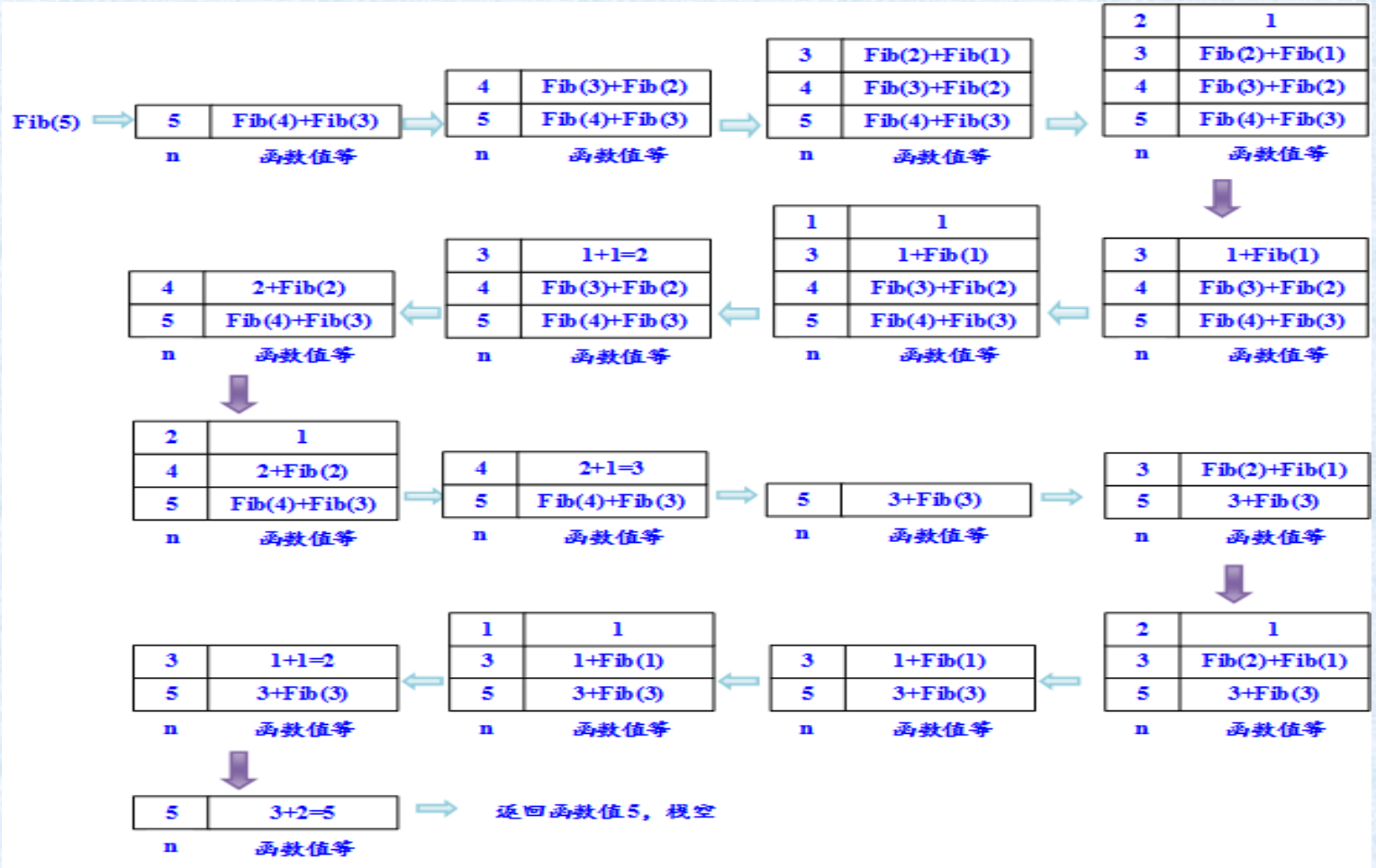
画出求Fib(5)的递归树以及递归工作栈的变化和求解过程。

解：求Fib(5)的递归树如下：



从上面求Fib(5)的过程看到，对于复杂的递归调用，分解和求值可能交替进行、循环反复，直到求出最终值。

执行Fib(5)时递归工作栈的变化和求解过程:



2.2 递归算法设计

2.2.1 递归与数学归纳法

第一数学归纳法原理：若{P(1)，P(2)，P(3)，P(4)，...}是命题序列且满足以下两个性质，则所有命题均为真：

（1）P(1)为真。

（2）任何命题均可以从它的前一个命题推导得出。

例如，采用第一数学归纳法证明下式：

1+2+…+n=$\frac{n(n+1)}{2}$

证明：

当n=1时，左式=1，右式=$\frac{1\times 2}{2}$ =1，左右两式相等，

等式成立。

假设当n=k-1时等式成立，有1+2+…+(k-1)=$\frac{k(k-1)}{2}$

当n=k时，左式=1+2+…+k=1+2+…+(k-1)+k $\frac{k(k-1)}{2}$ +k $\frac{k(k+1)}{2}$

等式成立。即证。

第二数学归纳法原理：若{P(1)，P(2)，P(3)，P(4)，...}是满足以下两个性质的命题序列，则对于其他自然数，该命题序列均为真：

（1）P(1)为真。

（2）任何命题均可以从它的前面所有命题推导得出。

归纳步骤（条件2）的意思是P(n)可以从前面所有命题假设{P(1)，P(2)，P(3)，...，P(n-1)}推导得出。

例如，采用第二数学归纳法证明，任何含有n（n≥0）个不同结点的二叉树，都可由它的中序序列和先序序列唯一地确定。

证明：当n=0时，二叉树为空，结论正确。

假设结点数小于n的任何二叉树（所有结点值不相同），都可以由其先序序列和中序序列唯一地确定。

根据归纳假设，由于子先序序列a1…ak和子中序序列b0b1…bk-1可以唯一地确定根结点a0的左子树，而子先序序列ak+1…an-1和子中序序列bk+1…bn-1可以唯一地确定根结点a0的右子树。

综上所述，这棵二叉树的根结点已经确定，而且其左、右子树都唯一地确定了，所以整个二叉树也就唯一地确定了。

数学归纳法是一种论证方法，而递归是算法和程序设计的一种实现技术，数学归纳法是递归的基础。

2.2.1 递归算法设计的一般步骤

递归算法设计先要给出递归模型，再转换成对应的C/C++语言函数，获取递归模型的步骤如下：

（1）对原问题f(sn)进行分析，抽象出合理的“小问题”f(sn-1)（与数学归纳法中假设n=k-1时等式成立相似）；

（2）假设f(sn-1)是可解的，在此基础上确定f(sn)的解，即给出f(sn)与f(sn-1)之间的关系（与数学归纳法中求证n=k时等式成立的过程相似）；

（3）确定一个特定情况（如f(1)或f(0)）的解，由此作为递归出口（与数学归纳法中求证n=1或n=0时等式成立相似）。

【例2.5】用递归法求一个整数数组a的最大元素。

解：设f(a，i)求解数组a中前i个元素即a[0..i-1]中的最大元素，则f(a，i-1)求解数组a中前i-1个元素即a[0..i-2]中的最大元素，前者为“大问题”，后者为“小问题”。

假设f(a，i-1)已求出，则有f(a，i)=MAX{f(a，i-1)，a[i-1]}。递推方向是朝a中元素减少的方向推进，当a中只有一个元素时，该元素就是最大元素，所以f(a，1)=a[0]。

由此得到递归模型如下：

$f(a, i)=a[0]$　　当$i=1$时

$f(a, i)=\mathrm{MAX}\{f(a, i-1), a[i-1]\}$　　当$i>1$时

对应的递归算法如下：

```
int fmax(int a[], int i)
{   if (i==1)
            return a[0];
    else
            return(fmax(a, i-1), a[i-1]);
}
```

2.2.3 递归数据结构及其递归算法设计

1. 递归数据结构的定义

采用递归方式定义的数据结构称为递归数据结构。在递归数据结构定义中包含的递归运算称为基本递归运算。

归纳起来，递归数据结构定义为：

RD=(D，Op)

其中，D={di}（1≤i≤n，共n个元素）为构成该数据结构的所有元素的集合。

Op是基本递归运算的集合，Op={opj}（1≤j≤m，共m个基本递归运算），对于∀di∈D，不妨设opj为一元运算符，则有opj(di)∈D，也就是说，递归运算符具有封闭性。

二叉树的定义中，D是给定二叉树及其子树的集合（对于一棵给定的二叉树，其子树的个数是有限的），Op={op1，op2}由基本递归运算符构成，它们的定义如下：

op1(p) = p->lchild

op2(p) = p->rchild

其中，p指向二叉树中的一个非空结点。

2. 基于递归数据结构的递归算法设计

1）单链表的递归算法设计

在设计不带头结点的单链表的递归算法时：

设求解以L为首结点指针的整个单链表的某功能为“大问题”。

而求解除首结点外余下结点构成的单链表（由L->next标识，而该运算为递归运算）的相同功能为“小问题”。

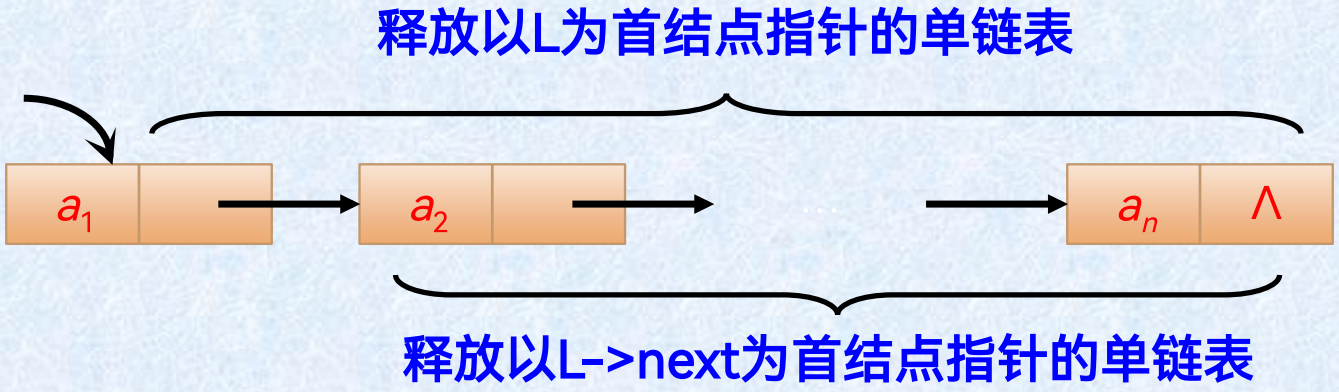
由大小问题之间的解关系得到递归体。

再考虑特殊情况，通常是单链表为空或者只有一个结点时，这时很容易求解，从而得到递归出口。

【例2.6】有一个不带头结点的单链表L，设计一个算法释放其中所有结点。

解：设L={a1，a2，…，an}，f(L)的功能是释放a1～an的所有结点，则f(L->next)的功能是释放a2～an的所有结点，前者是“大问题”，后者是“小问题”。

假设f(L->next)是已实现，则f(L)就可以采用先调用f(L->next)，然后释放L所指结点来求解。



对应的递归模型如下：

f(L) ≡不做任何事件　　当L=NULL时
f(L) ≡ f(L->next); 释放*L结点　　其他情况

```
void DestroyList(LinkNode *&L)
//释放单链表L中所有结点
{  if (L!=NULL)
   {      DestroyList(L->next);
          free(L);
   }
}
```

2）二叉树的递归算法设计

二叉树是一种典型的递归数据结构，当一棵二叉树采用二叉链b存储时：

设求解以b为根结点的整个二叉树的某功能为“大问题”。

求解其左、右子树的相同功能为“小问题”。

由大小问题之间的解关系得到递归体。

再考虑特殊情况，通常是二叉树为空或者只有一个结点时，这时很容易求解，从而得到递归出口。

【例2.8】对于含n（n>0）个结点的二叉树，所有结点值为int类型，设计一个算法由其先序序列a和中序序列b创建对应的二叉链存储结构。

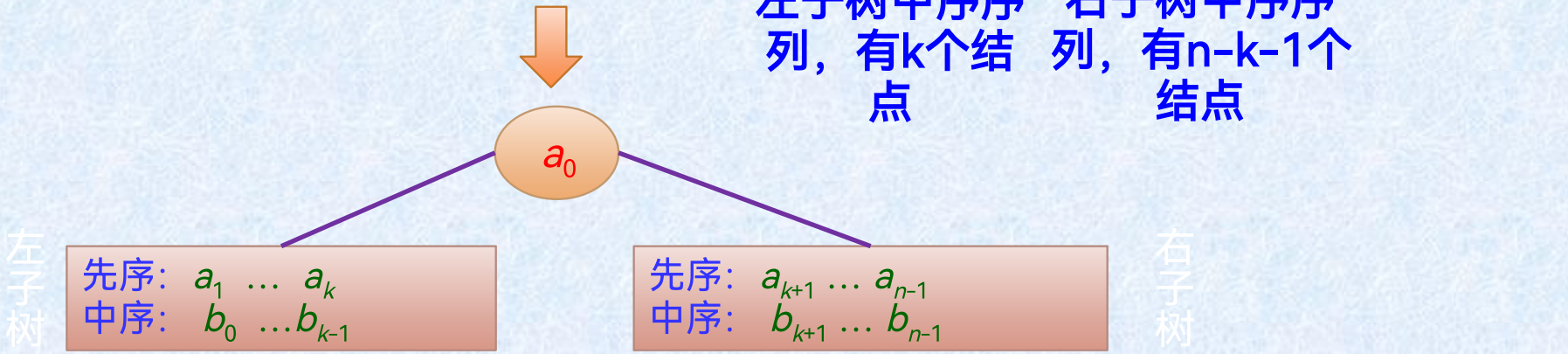
通过根结点a0在中序序列中找到bk

先序序列：$a_0\ a_1 \ldots a_k\ a_{k+1} \ldots a_{n-1}$

左子树先序序列，有k个结点　右子树先序序列，有n-k-1个结点

$b_0\ b_1 \ldots b_{k-1}\ b_k\ b_{k+1} \ldots b_{n-1}$　中序序列

左子树中序序列，有k个结点　右子树中序序列，有n-k-1个结点



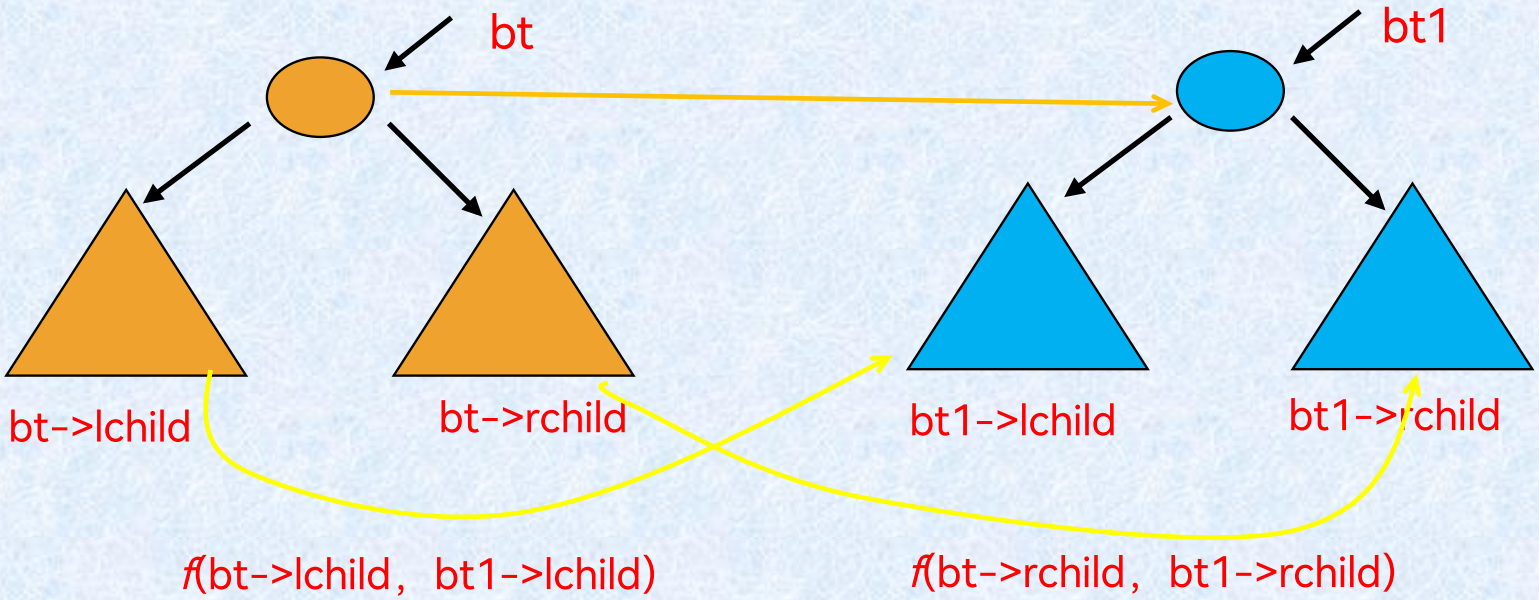
```
BTNode *CreateBTree(ElemType a[],ElemType b[],int n)
//由先序序列a[0..n-1]和中序序列b[0..n-1]建立二叉链存储结构bt
{   int k;
    if (n<=0) return NULL;
    ElemType root=a[0];                                    //根结点值
    BTNode *bt=(BTNode *)malloc(sizeof(BTNode));
    bt->data=root;
    for (k=0;k<n;k++)                          //在b中查找b[k]=root的根结点
      if (b[k]==root)
            break;

    bt->lchild=CreateBTree(a+1,b,k);                  //递归创建左子树
    bt->rchild=CreateBTree(a+k+1,b+k+1,n-k-1);  //递归创建右子树

    return bt;
}
```

【例2.10】假设二叉树采用二叉链存储结构，设计一个递归算法由二叉树bt复制产生另一棵二叉树bt1。

解：设f(bt，bt1)的功能是由二叉树bt复制产生另一棵二叉树bt1，它是“大问题”，则f(bt->lchild，bt1->lchild)的功能就是由bt的左子树复制产生bt1的左子树，f(bt->rchild，bt1->rchild)的功能就是由bt的右子树复制产生bt1的右子树，它们是“小问题”。



对应的递归模型如下：

f(bt，bt1) ≡ bt1=NULL　　当b=NULL时
f(bt，bt1) ≡ 由bt结点复制产生bt1结点;　　其他情况
　　f(bt->lchild，bt1->lchild);
　　f(bt->rchild，bt1->rchild)

```
void CopyBTree(BTNode *bt,BTNode *&bt1)
//由二叉树bt复制产生bt1
{  if (bt==NULL)
        bt1=NULL;
   else
   {    bt1=(BTNode *)malloc(sizeof(BTNode));
        bt1->data=bt->data;
        CopyBTree(bt->lchild,bt1->lchild);
        CopyBTree(bt->rchild,bt1->rchild);
   }
}
```

【例2.11】假设二叉树采用二叉链存储结构，设计一个递归算法输出从根结点到值为x的结点的路径，假设二叉树中所有结点值不同。

解法：用vector<int>向量path存放从根结点到x结点的正向路径。

f(b，x，path)的求解过程是：

若b为空树，返回false；

否则将b结点值加入到path中，如果b->data=x，查找成功返回true；

如果b->data≠x，在左子树中查找，若在左子树找到值为x的结点，返回true，若在左子树没有找到值为x的结点，返回在右子树中的查找结果。

左、右子树中查找都是“小问题”。

f(b，x，path)=false　　当b=NULL

f(b，x，path)=true（将b->data加入到path中）　　当b->data=x

f(b，x，path)=true　　f(b->lchild，x，path)=true

f(b，x，path)= f(b->rchild，x，path)　　其他情况

```
bool Findxpath2(BTNode *bt,int x,vector<int> tmppath,
    vector<int> &path)                    //求根结点到x结点的（正向）路径
{   if (bt==NULL)                         //空树返回false
        return false;
    tmppath.push_back(bt->data);   //当前结点加入path
    if (bt->data==x)                              //当前结点值为x，返回true
    {       path=tmppath;                         //路径复制
        return true;
    }
    bool find=Findxpath2(bt->lchild,x,tmppath,path); //在左子树中查找
    if (find)                                     //左子树中成功找到
        return true;
    else                                  //左子树中没有找到，在右子树中查找
        return Findxpath2(bt->rchild,x,tmppath,path);
}
```

2.3 递归算法设计示例

2.3.1 简单选择排序和冒泡排序

【问题描述】对于给定的含有n个元素的数组a，分别采用简单选择排序和冒泡排序方法对其按元素值递增排序。

1. 简单选择排序

设f(a，n，i)用于对a[i..n-1]元素序列（共n-i个元素）进行简单选择排序，是“大问题”.

f(a，n，i+1)用于对a[i+1..n-1]元素序列（共n-i-1个元素）进行简单选择排序，是“小问题”。

当i=n-1时所有元素有序，算法结束。

f(a，n，i) ≡ 不做任何事情，算法结束当i=n-1
f(a，n，i) ≡ 通过简单比较挑选a[i..n-1]中的最小元素a[k]放在a[i]处;
否则f(a，n，i+1);

```
void SelectSort(int a[]，int n，int i)
{   int j，k;
    if (i==n-1) return;   //满足递归出口条件

    else
      {   k=i;                          //k记录a[i..n-1]中最小元素的下标
          for (j=i+1;j<n;j++)       //在a[i..n-1]中找最小元素
                if (a[j]<a[k])
                    k=j;
          if (k!=i)             //若最小元素不是a[i]
             swap(a[i],a[k]);       //a[i]和a[k]交换

          SelectSort(a，n，i+1);
      }
}
```

2. 冒泡排序

设f(a，n，i)用于对a[i..n-1]元素序列（共n-i个元素）进行冒泡排序，是“大问题”，则f(a，n，i+1)用于对a[i+1..n-1]元素序列（共n-i-1个元素）进行冒泡排序，是“小问题”。当i=n-1时所有元素有序，算法结束。

f(a，n，i) ≡ 不做任何事情，算法结束　　当i=n-1

f(a，n，i) ≡ 对a[i..n-1]元素序列，从a[n-1]开始

进行相邻元素比较;　　否则

若相邻两元素反序则将两者交换;

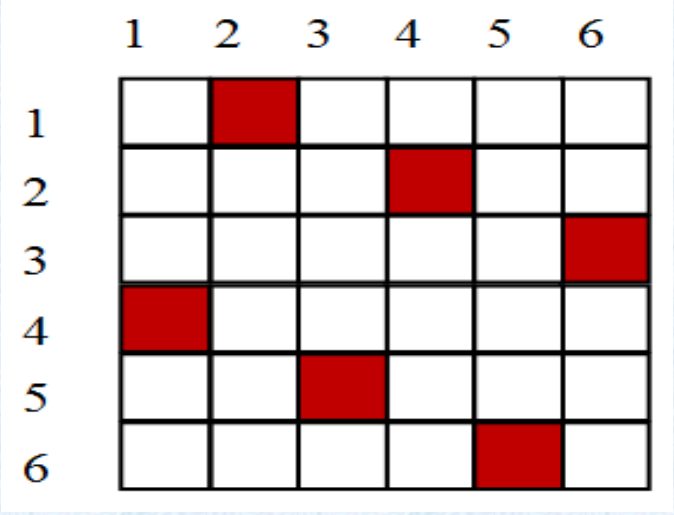
若没有交换则返回，否则执行f(a，n，i+1);

```
void BubbleSort(int a[], int n, int i)
{  int j;
   bool exchange;
   if (i==n-1) return;                    //满足递归出口条件

      else
      {  exchange=false;                  //置exchange为false
         for (j=n-1;j>i;j--)
               if (a[j]<a[j-1])                    //当相邻元素反序时
               {       swap(a[j],a[j-1]);
                       exchange=true;              //发生交换置exchange为true
               }
          if (exchange==false)                     //未发生交换时直接返回
               return;
          else                                     //发生交换时继续递归调用
               BubbleSort(a, n, i+1);
      }
}
```

2.3.2 求解n皇后问题

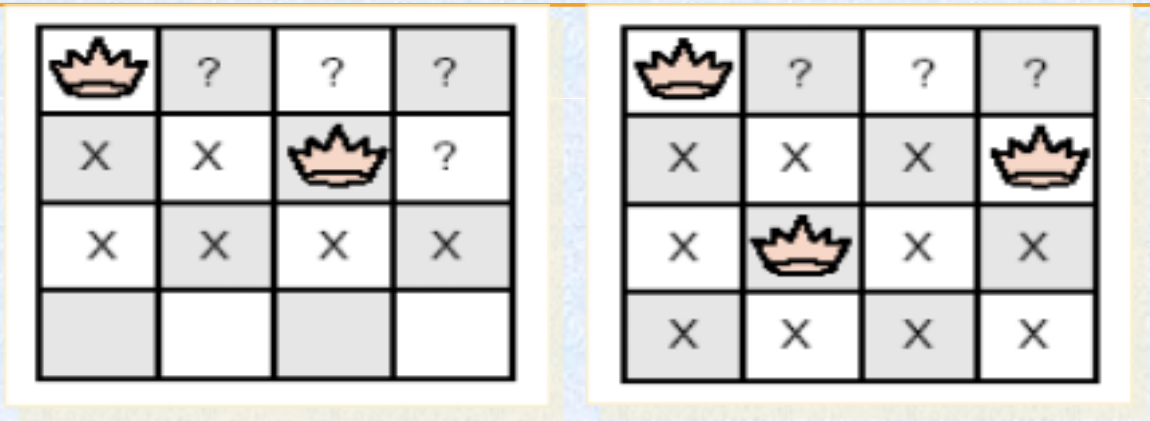
【问题描述】在n×n的方格棋盘上，放置n个皇后，要求每个皇后不同行、不同列、不同左右对角线。如下图所示是6皇后问题的一个解。



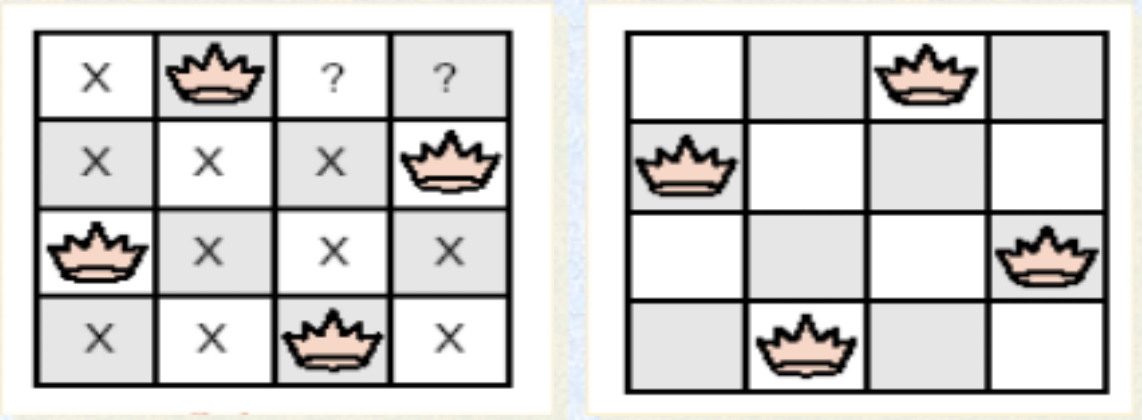
q[1..6]={2，4，6，1，3，5}

Example

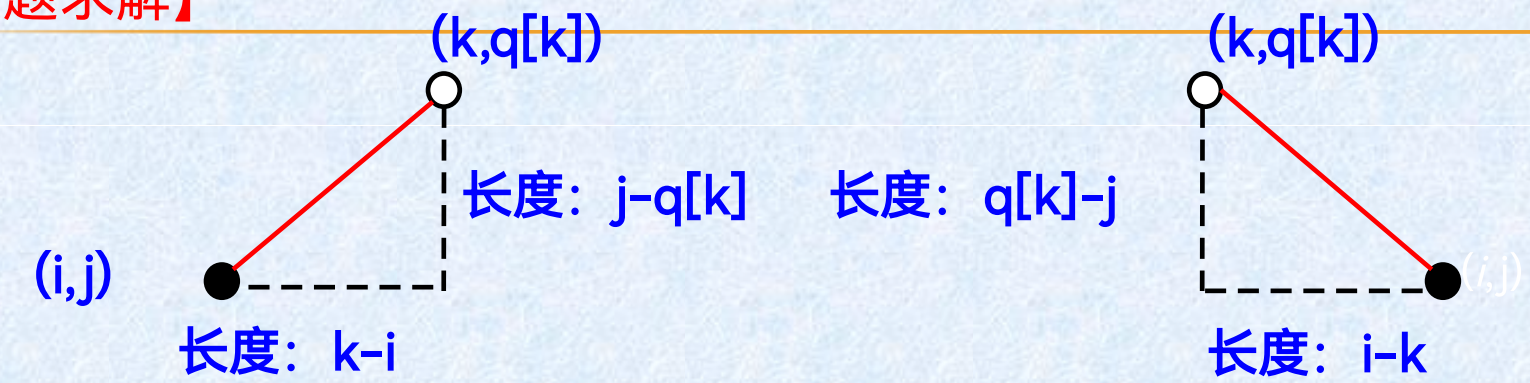
Four queens



Dead end Solution



【问题求解】



对于(i，j)位置上的皇后，是否与已放好的皇后(k，q[k])（1≤k≤i-1）有冲突呢?

- 显然它们不同列，若同列则有：q[k]==j;
- 对角线有两条，若它们在任一条对角线上，则构成一个等边直角三角形，即|q[k]-j|==|i-k|。

(q[k]==j) || (abs(q[k]-j)==abs(i-k))

设queen(i，n)是在1～i-1行上已经放好了i-1个皇后，用于在i～n行放置n-i+1个皇后，则queen(i+1，n)表示在1～i行上已经放好了i个皇后，用于在i+1～n行放置n-i个皇后。

queen(i+1，n)比queen(i，n)少放置一个皇后。所以queen(i+1，n)是“小问题”，queen(i，n)是“大问题”。

queen(i，n) ≡ n个皇后放置完毕，输出一个解　　若k>n

queen(i，n) ≡ 在第i行的合适的位置（i,j）　其他情况

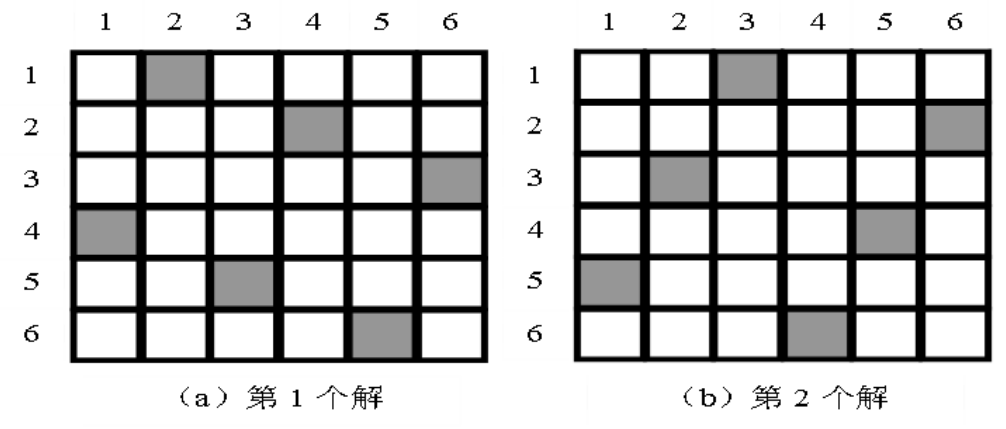
　　在其上放置一个皇后;

queen(i+1，n);

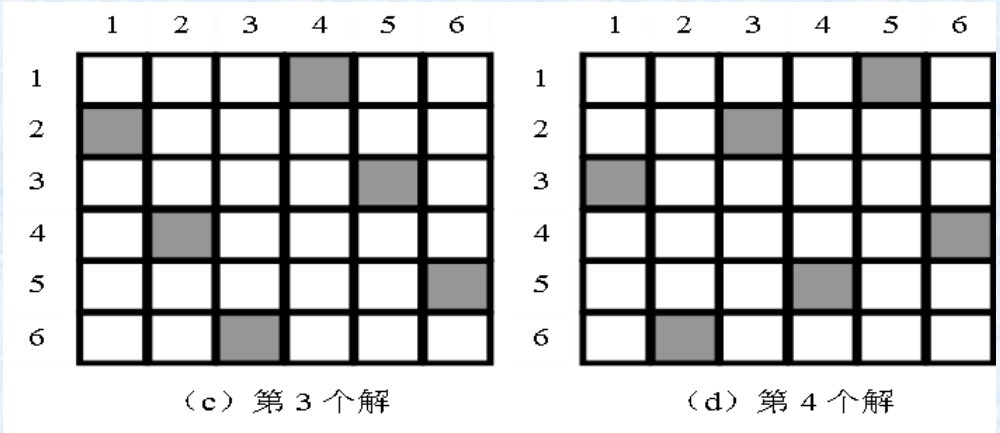
```
bool place(int i,int j)                    //测试(i,j)位置能否摆放皇后
{   if (i==1) return true;                 //第一个皇后总是可以放置
    int k=1;
    while (k<i)                            //k=1 ~ i-1是已放置了皇后的行
    {       if ((q[k]==j) || (abs(q[k]-j)==abs(i-k)))
                return false;
            k++;
    }
    return true;
}
void queen(int i,int n)                    //放置1 ~ i的皇后
{   if (i>n)
            dispasolution(n);              //所有皇后放置结束
    else
    {       for (int j=1;j<=n;j++)         //在第i行上试探每一个列j
                if (place(i,j))            //在第i行上找到一个合适位置(i,j)
                {       q[i]=j;
                        queen(i+1,n);
                }
    }
}
```

本程序一次执行结果如下:

皇后问题(n<20) n=6↙
6皇后问题求解如下:
第1个解：(1，2) (2，4) (3，6) (4，1) (5，3) (6，5)
第2个解：(1，3) (2，6) (3，2) (4，5) (5，1) (6，4)



第3个解：(1，4) (2，1) (3，5) (4，2) (5，6) (6，3)
第4个解：(1，5) (2，3) (3，1) (4，6) (5，4) (6，2)



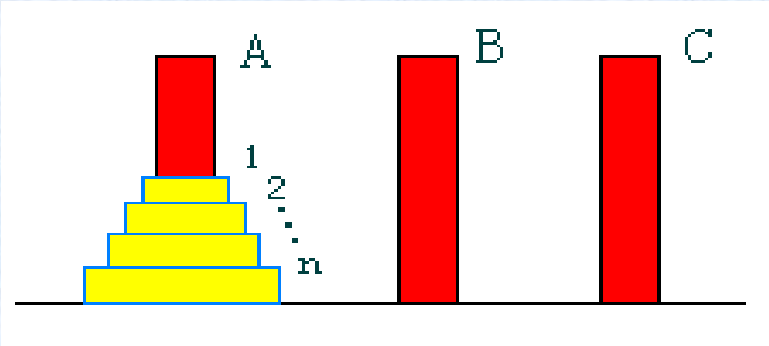
2.3.2 汉诺塔问题

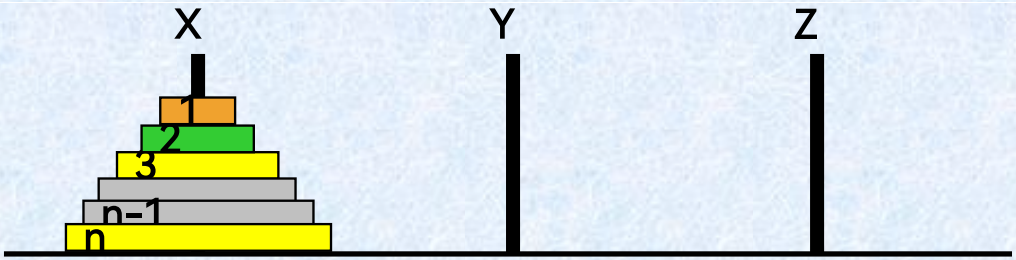
设a,b,c是3个塔座。开始时，在塔座a上有一叠共n个圆盘，这些圆盘自下而上，由大到小地叠在一起。各圆盘从小到大编号为1,2,…,n,现要求将塔座a上的这一叠圆盘移到塔座b上，并仍按同样顺序叠置。在移动圆盘时应遵守以下移动规则：

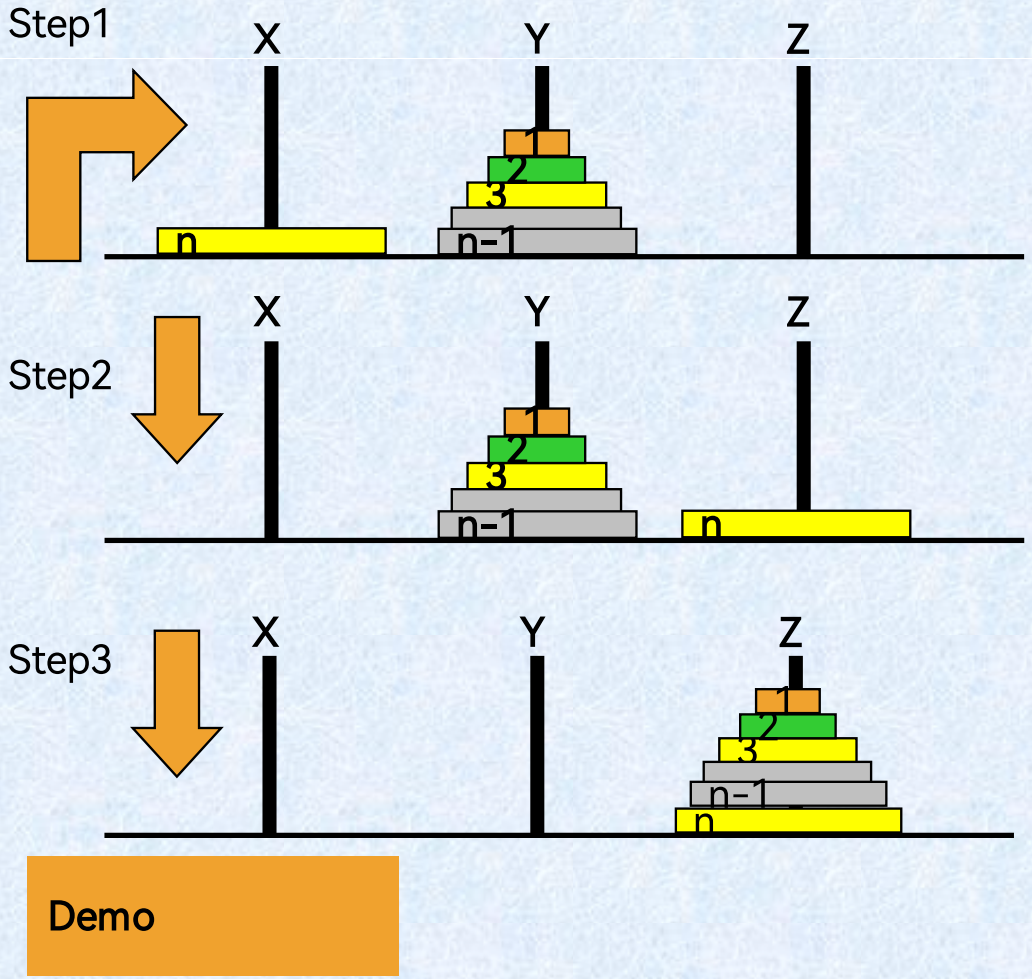
规则1：每次只能移动1个圆盘；

规则2：任何时刻都不允许将较大的圆盘压在较小的圆盘之上；

规则3：在满足移动规则1和2的前提下，可将圆盘移至a,b,c中任一塔座上。




X
Y
Z
1
2
3
n−1
n


Step1
X
Y
Z
1
2
3
n−1
n
Step2
X
Y
Z
1
2
3
n−1
n
Step3
X
Y
Z
1
2
3
n−1
n

Demo

METHOD SPECIFICATION

void move(**int** count, **int** start, **int** finish, **int** temp);

Pre: There are at least count disks on the tower start. The top disk (if any) on each of towers temp and finish is larger than any of the top count disks on tower start.

Post: The top count disks on start have been moved to finish; temp (used for temporary storage) has been returned to its starting position.

PROGRAM FOR HANOI

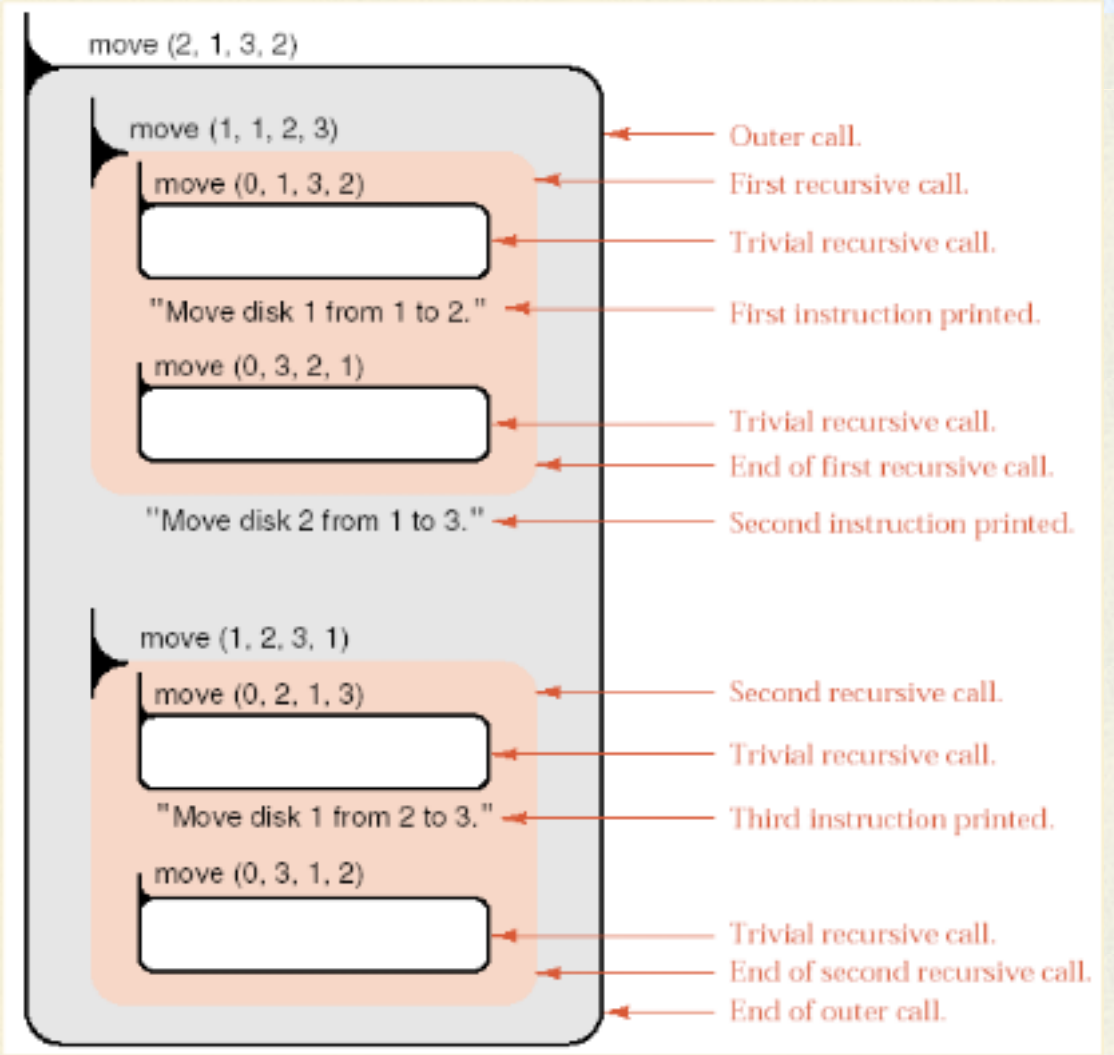
```
const int disks = 64;
 // Make this constant much smaller to run program.
void move(int count, int start, int finish, int temp);
/*
Pre:  None.
Post: The simulation of the Towers of Hanoi has terminated.
*/
main()
{
   move(disks, 1, 3, 2);
}
```

HANOI RECURSIVE FUNCTION

```
void move(int count, int start, int finish, int temp)
1:{
2:    if (count > 0) {
3:       move(count - 1, start, temp, finish);
4:       cout << "Move disk " << count << " from " << start
<< " to " << finish << "." << endl;
5:       move(count - 1, temp, finish, start);
6:    }
7:}
```

Example:

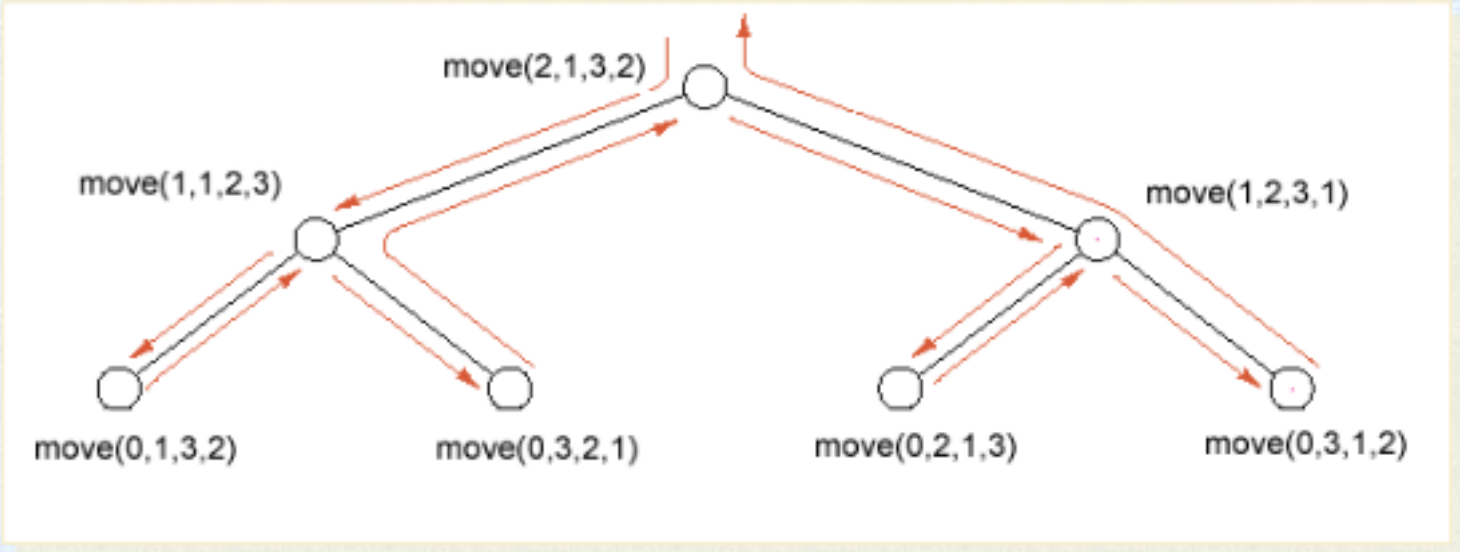
move(2, 1, 3, 2)



Recursion tree of Hanoi

Example:

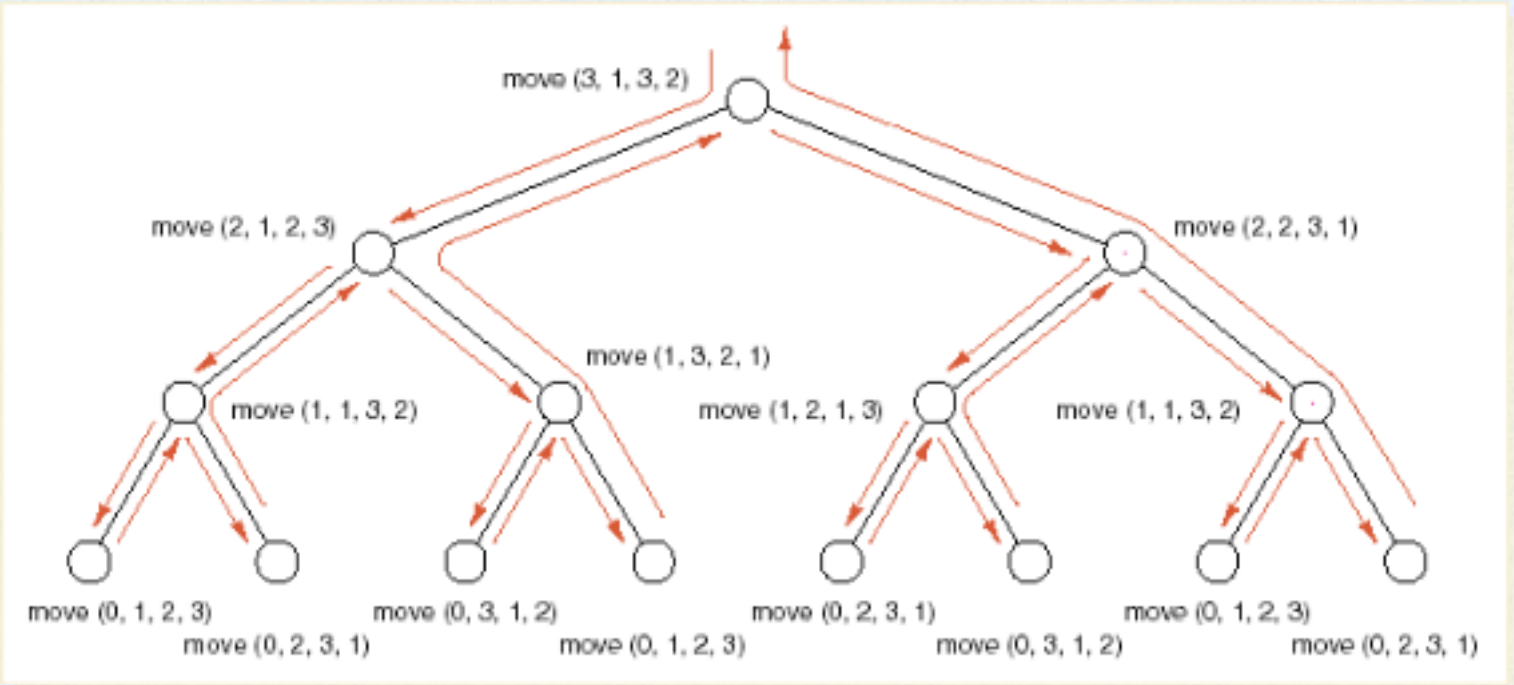
move(2, 1, 3, 2)



Recursion tree of Hanoi

Example:

move(3, 1, 3, 2)



小结

递归算法的优缺点：

优点：结构清晰，可读性强，而且容易用数学归纳法来证明算法的正确性，因此它为设计算法、调试程序带来很大方便。

缺点：递归算法的运行效率较低，无论是耗费的计算时间还是占用的存储空间都比非递归算法要多。

2.4 分治法的设计思想

对于一个规模为n 的问题：若该问题可以容易地解决（比如说规模n 较小）则直接解决，否则将其分解为k个规模较小的子问题，这些子问题互相独立且与原问题形式相同，递归地解这些子问题，然后将各子问题的解合并得到原问题的解。

这种算法设计策略叫做分治法。

分治法所能解决的问题一般具有以下几个特征

（1）该问题的规模缩小到一定的程度就可以容易地解决。

（2）该问题可以分解为若干个规模较小的相同问题。

（3）利用该问题分解出的子问题的解可以合并为该问题的解。

（4）该问题所分解出的各个子问题是相互独立的，即子问题之间不包含公共的子问题。

分治法通常采用递归算法设计技术，在每一层递归上都有3个步骤：

① 分解：将原问题分解为若干个规模较小，相互独立，与原问题形式相同的子问题。

② 求解子问题：若子问题规模较小而容易被解决则直接求解，否则递归地求解各个子问题。

③ 合并：将各个子问题的解合并为原问题的解。

分治法的一般的算法设计框架如下：

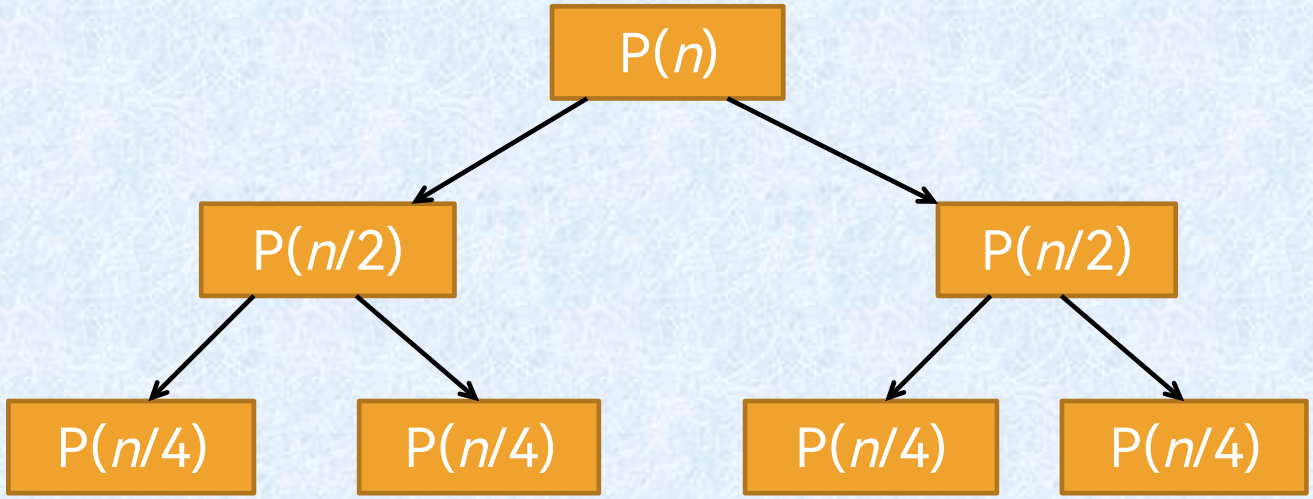
```
divide-and-conquer(P)
{  if |P|≤n0 return adhoc(P);
   将P分解为较小的子问题 P1，P2，…，Pk;
   for(i=1;i<=k;i++)                    //循环处理k次
      yi=divide-and-conquer(Pi);        //递归解决Pi
       return merge(y1，y2，…，yk); //合并子问题
}
```

根据分治法的分割原则，原问题应该分为多少个子问题才较适宜？各个子问题的规模应该怎样才为适当？

这些问题很难予以肯定的回答。但人们从大量实践中发现，在用分治法设计算法时，最好使子问题的规模大致相同。换句话说，将一个问题分成大小相等的k个子问题的处理方法是行之有效的。

当k=1时称为减治法。

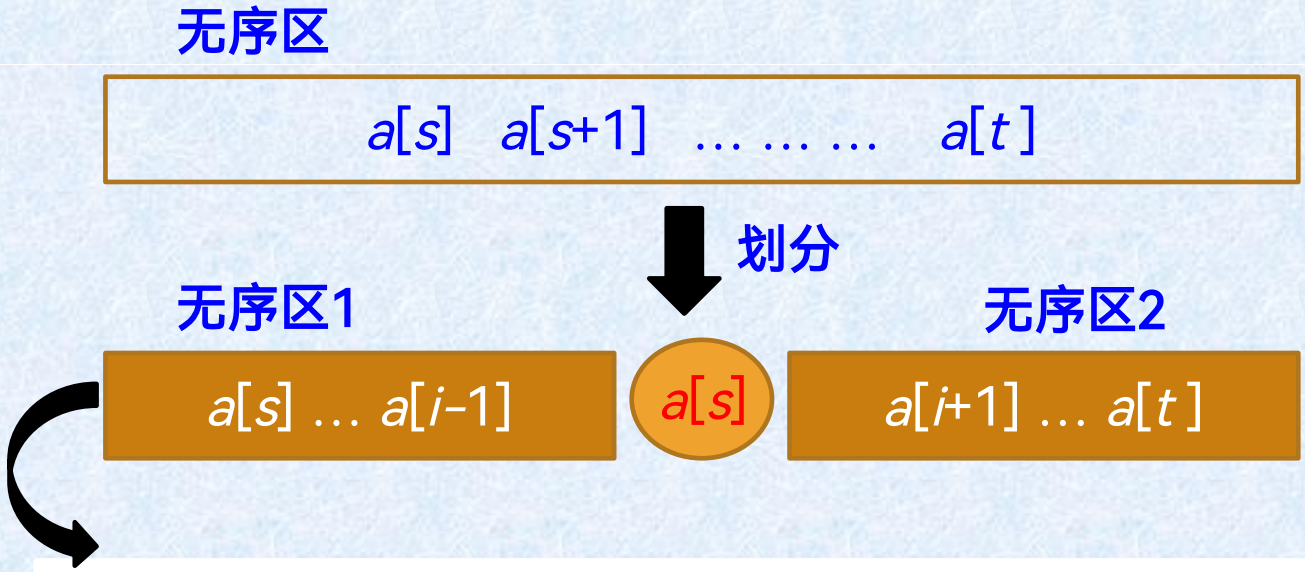
许多问题可以取 k=2，称为二分法，如图4.1所示，这种使子问题规模大致相等的做法是出自一种平衡子问题的思想，它几乎总是比子问题规模不等的做法要好。



2.5 快速排序

基本思想：在待排序的n个元素中任取一个元素（通常取第一个元素）作为基准，把该元素放入最终位置后，整个数据序列被基准分割成两个子序列，所有小于基准的元素放置在前子序列中，所有大于基准的元素放置在后子序列中，并把基准排在这两个子序列的中间，这个过程称作划分。

然后对两个子序列分别重复上述过程，直至每个子序列内只有一个记录或空为止。



f (a,s,t) ≡ 不做任何事情　　当a[s..t]中长度小于2

f (a,s,t) ≡ i=Partition(a,s,t);其他情况

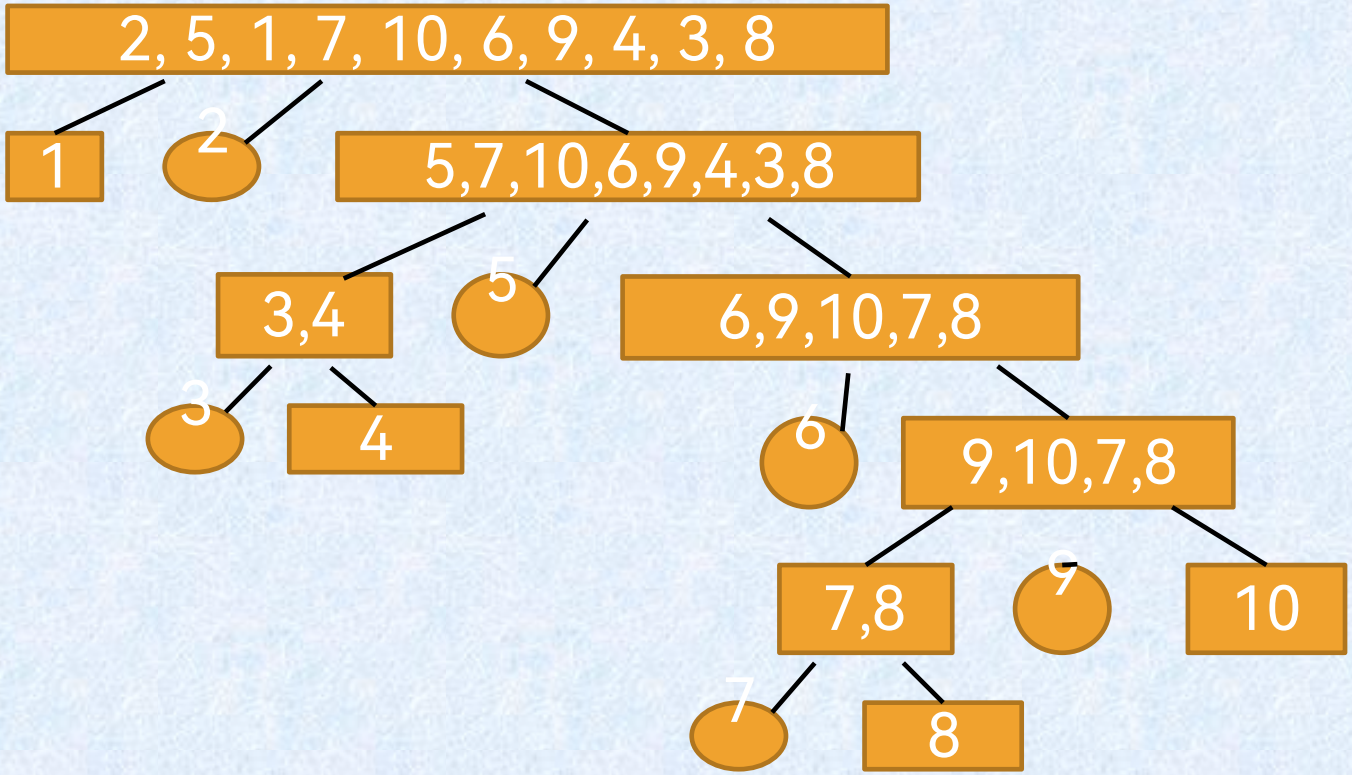
```
        f (a,s,i-1);
        f (a,i,t );
```

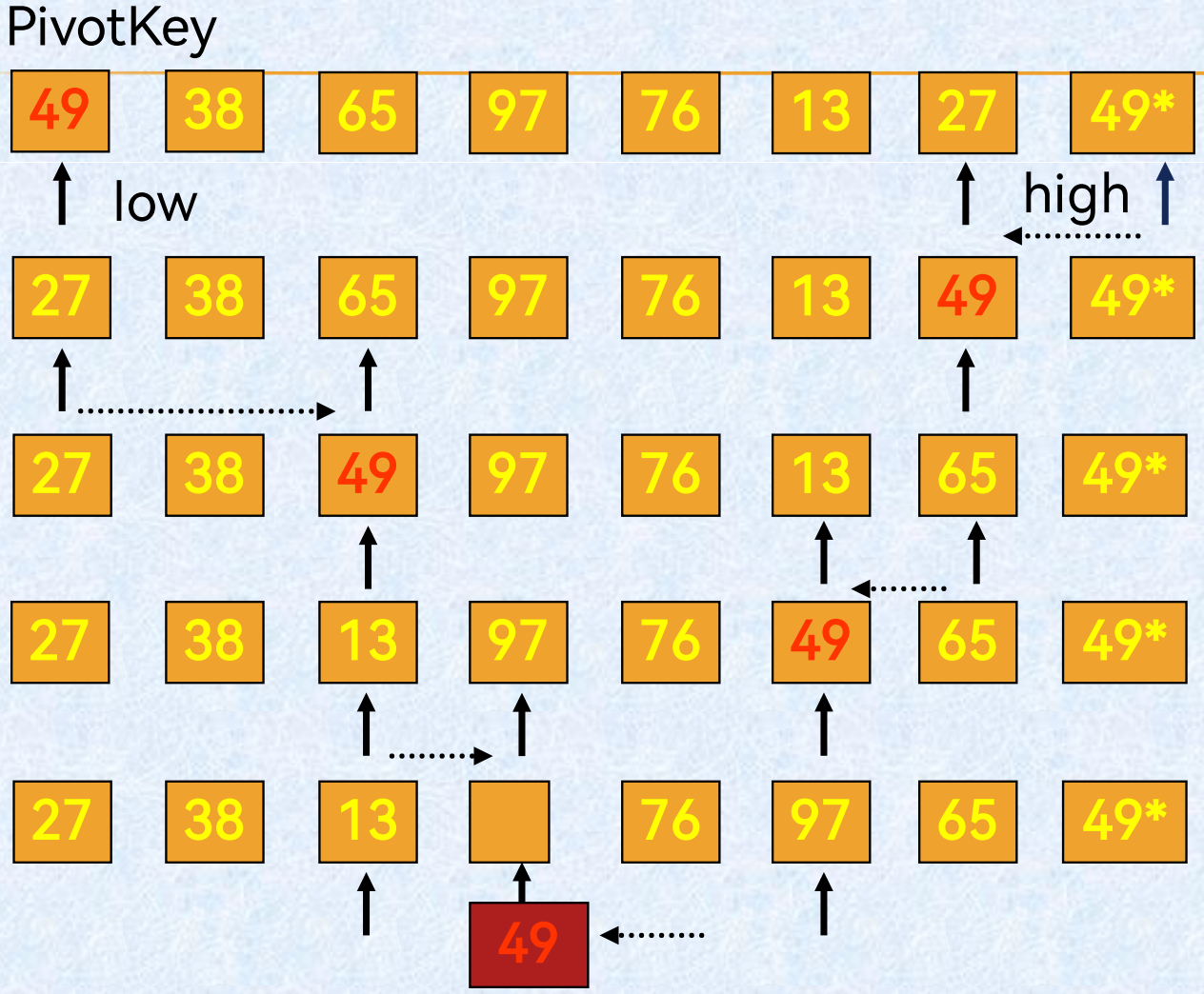
① 分解：将原序列a[s..t]分解成两个子序列a[s..i-1]和a[i+1..t]，其中i为划分的基准位置。

② 求解子问题：若子序列的长度为0或为1，则它是有序的，直接返回；否则递归地求解各个子问题。

③ 合并：由于整个序列存放在数组中a中，排序过程是就地进行的，合并步骤不需要执行任何操作。

例如，对于{2，5，1，7，10，6，9，4，3，8}序列，其快速排序过程如下图所示。




PivotKey
49
38
65
97
76
13
27
49*
low
high
27
38
65
97
76
13
49
49*
27
38
49
97
76
13
65
49*
27
38
13
97
76
49
65
49*
27
38
13
76
97
65
49*
49

```
int Partition(int a[]，int s，int t)      //划分算法
{   int i=s，j=t;
    int tmp=a[s];          //用序列的第1个记录作为基准
    while (i!=j)   //从序列两端交替向中间扫描，直至i=j为止
    {   while (j>i && a[j]>=tmp)
            j--;//从右向左扫描，找第1个关键字小于tmp的a[j]
        a[i]=a[j]; //将a[j]前移到a[i]的位置
        while (i<j && a[i]<=tmp)
            i++;//从左向右扫描，找第1个关键字大于tmp的a[i]
        a[j]=a[i]; //将a[i]后移到a[j]的位置
       }
       a[i]=tmp;
       return i;
}
```

快速排序算法

```
void QuickSort(int a[ ]，int s，int t)
//对a[s..t]元素序列进行递增排序
{  if (s<t)          //序列内至少存在2个元素的情况
   {   int i=Partition(a，s，t);
       QuickSort(a，s，i-1);    //对左子序列递归排序
       QuickSort(a，i+1，t);    //对右子序列递归排序
   }
}
```

【算法分析】快速排序的时间主要耗费在划分操作上，对长度为n的区间进行划分，共需n-1次关键字的比较，时间复杂度为O(n)。

对n个记录进行快速排序的过程构成一棵递归树，在这样的递归树中，每一层至多对n个记录进行划分，所花时间为O(n)。

当初始排序数据正序或反序时，此时的递归树高度为n，快速排序呈现最坏情况，即最坏情况下的时间复杂度为$O(n^2)$;

【算法分析】当初始排序数据随机分布，使每次分成的两个子区间中的记录个数大致相等，此时的递归树高度为$\log_2 n$，快速排序呈现最好情况，即最好情况下的时间复杂度为$O(n\log_2 n)$。快速排序算法的平均时间复杂度也是$O(n\log_2 n)$。

2.6 主方法

主方法（master method）提供了解如下形式递归方程的一般方法：

$$T(n)=aT(n/b)+f(n) \qquad (2.11)$$

其中$a\geq 1$，$b>1$为常数，该方程描述了算法的执行时间，算法将规模为n的问题分解成a个子问题，每个子问题的大小为n/b。

例如，对于递归方程$T(n)=3T(n/4)+n^2$，有：$a=3$，$b=4$，$f(n)=n^2$。

主定理：设$a \geq 1$，$b>1$为常数，$f(n)$为一个函数，$T(n)$由（2.11）的递归方程定义，其中n为非负整数，则$T(n)$计算如下：

（1）若对某些常数$\varepsilon>0$，有$f(n)=O(n^{\log_b a-\varepsilon})$

那么$T(n)=O(n^{\log_b a})$

（2）若$f(n)=O(n^{\log_b a})$，那么$T(n)=O(n^{\log_b a}\log_2 n)$。

（3）若对某些常数$\varepsilon>0$，有$f(n)=O(n^{\log_b a+\varepsilon})$，并且对常数$c<1$与所有足够大的$n$，有$af(n/b)\leq cf(n)$，那么$T(n)=O(f(n))$。

应用该定理的过程是，首先把函数$f(n)$与函数进行比较，递归方程的解由这两个函数中较大的一个决定：

情况（1），函数$n^{\log_b a}$比函数$f(n)$更大，则$T(n)=O(n^{\log_b a})$

情况（2），函数$n^{\log_b a}$和函数$f(n)$一样大，则$T(n)$

=　　$O(n^{\log_b a}\log_2 n)$

情况（3），函数$n^{\log_b a}$比函数$f(n)$小，则$T(n)=O(f(n))$。

【例2.17】分析以下递归方程的时间复杂度：

$T(n)=1$　当$n=1$

$T(n)=4T(n/2)+n$　　当$n>1$

解：这里$a=4$，$b=2$，$f(n)=n$。

因此，$n^{\log_2 4}=n^2$，比$f(n)$大，满足情况（1），

所以$T(n)=O(n^{\log_b a})$

$=O(n^2)$。

【例2.18】采用主方法求例2.15递归方程的时间复杂度。

$T(n)=1$　当$n=1$

$T(n)=2T(n/2)+n^2$　　当$n>1$

解：这里a=2，b=2，f(n)=n^2。因此，　$n^{\log_2 2}$，比f(n)小，满足情况（3），所以T(n)=O(f(n)) =O(n^2)，与采用递归树的结果相同。

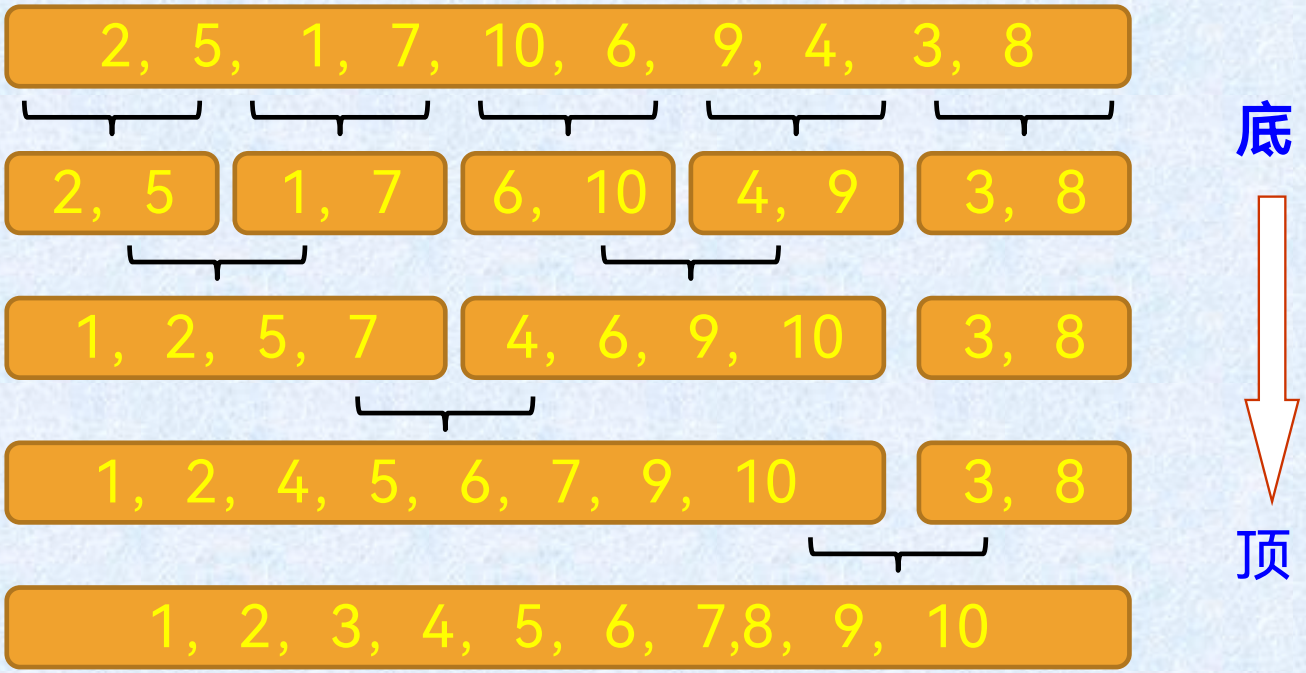
2.7 归并排序

归并排序的基本思想是：首先将$a[0..n-1]$看成是n个长度为1的有序表，将相邻的k（$k\geq 2$）个有序子表成对归并，得到n/k个长度为k的有序子表；然后再将这些有序子表继续归并，得到n/k^2个长度为k^2的有序子表，如此反复进行下去，最后得到一个长度为n的有序表。

若$k=2$，即归并在相邻的两个有序子表中进行的，称为二路归并排序。若$k>2$，即归并操作在相邻的多个有序子表中进行，则叫多路归并排序。

2.7.1自底向上的二路归并排序算法

例如，对于{2，5，1，7，10，6，9，4，3，8}序列，其排序过程如下图所示，图中方括号内是一个有序子序列。



二路归并排序的分治策略如下：

循环$\lceil \log_2 n \rceil$次，length依次取1、2、…、$\log_2 n$。每次执行以下步骤：

① 分解：将原序列分解成length长度的若干子序列。

② 求解子问题：将相邻的两个子序列调用Merge算法合并成一个有序子序列。

③ 合并

```
void Merge(int a[ ]，int low，int mid，int high)
//a[low..mid]和a[mid+1..high]→a[low..high]
{ int *tmpa;
  int i=low，j=mid+1，k=0;
  tmpa=(int *)malloc((high-low+1)*sizeof(int));//临时空间
  while (i<=mid && j<=high)
   if (a[i]<=a[j])              //将第1子表中的元素放入tmpa中
        { tmpa[k]=a[i]; i++; k++; }
        else                                  //将第2子表中的元素放入tmpa中
        { tmpa[k]=a[j];   j++; k++; }
  while (i<=mid)                              //将第1子表余下部分复制到tmpa
  { tmpa[k]=a[i]; i++; k++; }
  while (j<=high)                             //将第2子表余下部分复制到tmpa
  { tmpa[k]=a[j]; j++; k++; }
  for (k=0，i=low;i<=high;k++，i++) //将tmpa复制回a中
       a[i]=tmpa[k];
  free(tmpa);                                 //释放tmpa所占内存空间
}
```

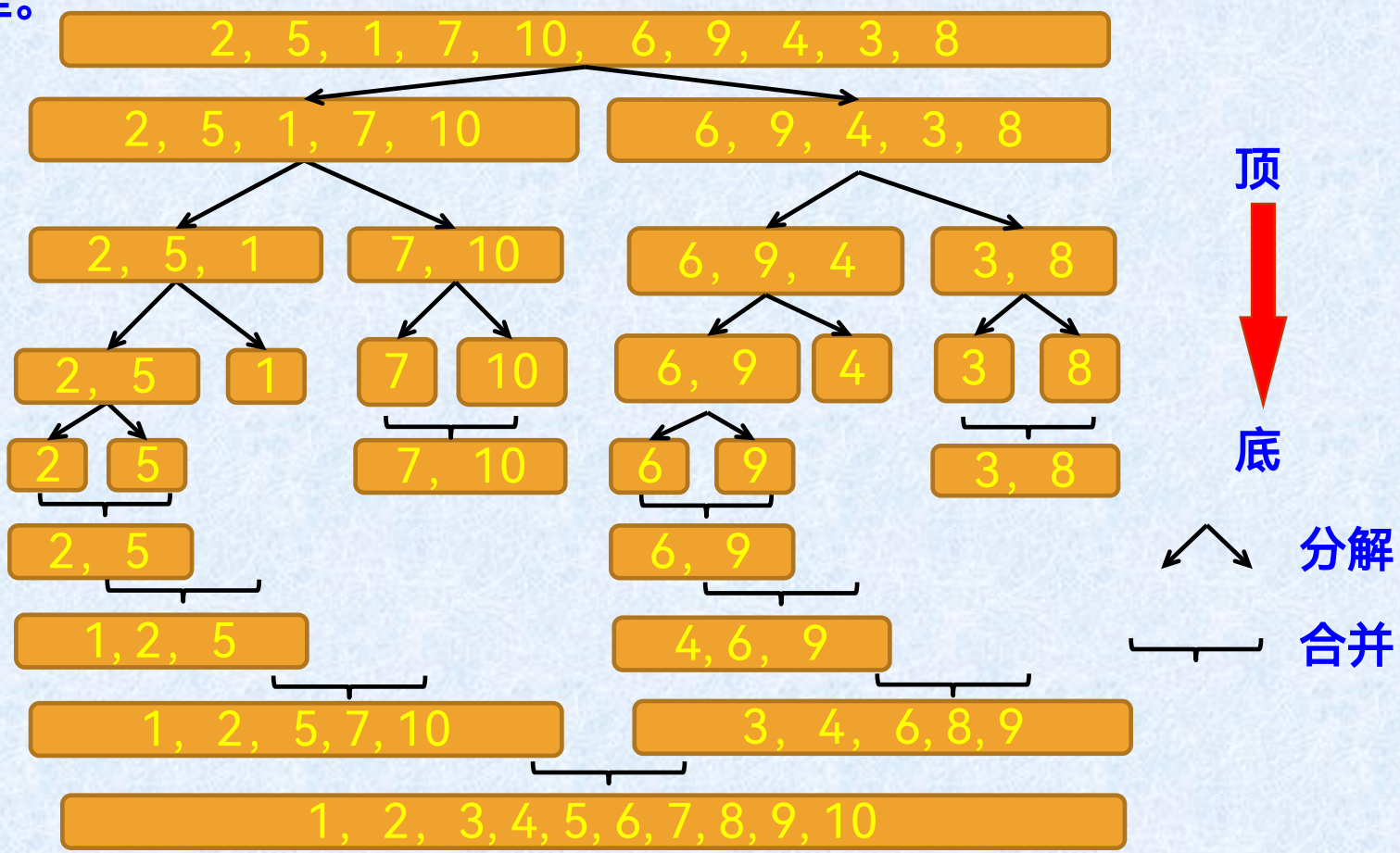
```
void MergePass(int a[ ]，int length，int n)
//一趟二路归并排序
{ int i;
  for (i=0;i+2*length-1<n;i=i+2*length) //归并length长的两相邻子表
        Merge(a，i，i+length-1，i+2*length-1);
        if (i+length-1<n)          //余下两个子表，后者长度小于length
        Merge(a，i，i+length-1，n-1); //归并这两个子表
}
```

```
void MergeSort(int a[ ]，int n)      //二路归并算法
{  int length;
   for (length=1;length<n;length=2*length)
            MergePass(a，length，n);
}
```

【算法分析】对于上述二路归并排序算法，当有n个元素时，需要$\lceil \log_2 n \rceil$趟归并，每一趟归并，其元素比较次数不超过$n-1$，元素移动次数都是n，因此归并排序的时间复杂度为$O(n\log_2 n)$。

2.7.1自顶向下的二路归并排序算法

例如，对于{2，5，1，7，10，6，9，4，3，8}序列，说明其自顶向下的二路归并排序的过程。



设归并排序的当前区间是*a*[low..high]，则递归归并的两个步骤如下：

① 分解：将序列*a*[low..high]一分为二，即求mid=(low+high)/2；递归地对两个子序列*a*[low..mid]和*a*[mid+1..high]进行继续分解。其终结条件是子序列长度为1（因为一个元素的子表一定是有序表）。

② 合并：与分解过程相反，将已排序的两个子序列*a*[low..mid]和*a*[mid+1..high]归并为一个有序序列*a*[low..high]。

对应的二路归并排序算法如下

```
void MergeSort(int a[ ]，int low，int high)
//二路归并算法
{   int mid;
      if (low<high)                          //子序列有两个或以上元素
      {     mid=(low+high)/2;                        //取中间位置
            MergeSort(a，low，mid); //对a[low..mid]子序列排序
            MergeSort(a，mid+1，high);         //对a[mid+1..high]子序列排序
            Merge(a，low，mid，high);//将两子序列合并，见前面的算法
      }
}
```

递归出口为序列长度为1或者0。

【算法分析】设MergeSort(a，0，n-1)算法的执行时间为$T(n)$，显然Merge(a，0，$n/2$，n-1)的执行时间为$O(n)$，所以得到以下递推式：

$T(n)=1$ 当$n=1$

$T(n)=2T(n/2)+O(n)$ 当$n>1$

容易推出，$T(n)=O(n\log_2 n)$。

2.8 寻找最大和次大元素

【问题描述】对于给定的含有n元素的无序序列，求这个序列中最大和次大的两个不同的元素。

例如：（2, 5, 1, 4, 6, 3），最大元素为6，次大元素为5。

【问题求解】对于无序序列*a*[low.high]中，采用分治法求最大元素max1和次大元素max2的过程如下：

（1）*a*[low.high]中只有一个元素：则max1=a[low]，max2=-INF（-∞）（要求它们是不同的元素）。

（2）*a*[low.high]中只有两个元素：则max1=MAX{a[low]，a[high]}，max2=MIN{a[low]，a[high]}。

（3）*a*[low.high]中有两个以上元素：按中间位置mid=(low+high)/2划分为a[low..mid]和a[mid+1..high]左右两个区间（注意左区间包含a[mid]元素）。

求出左区间最大元素lmax1和次大元素lmax2，求出右区间最大元素rmax1和次大元素rmax2。

合并：

若lmax1>rmax1，则max1=lmax1，max2=MAX{lmax2，rmax1};

否则max1=rmax1，max2=MAX{lmax1，rmax2}。

```
void solve(int a[ ],int low,int high,int &max1,int &max2)
{
    if (low==high)                  //区间只有一个元素
    {     max1=a[low]; max2=-INF; }
    else if (low==high-1)           //区间只有两个元素
    {     max1=max(a[low],a[high]); max2=min(a[low],
a[high]);
```

```
    else                        //区间有两个以上元素
    {   int mid=(low+high)/2;
        int lmax1,lmax2;
        solve(a,low,mid,lmax1,lmax2);           //左区间求lmax1和
lmax2
        int rmax1,rmax2;
        solve(a,mid+1,high,rmax1,rmax2);  //右区间求lmax1和
lmax2
        if (lmax1>rmax1)
        {   max1=lmax1;
            max2=max(lmax2,rmax1); //lmax2,rmax1中求次大元素
        }
        else
        {   max1=rmax1;
            max2=max(lmax1,rmax2); //lmax1,rmax2中求次大元素
        }
    }
}
```

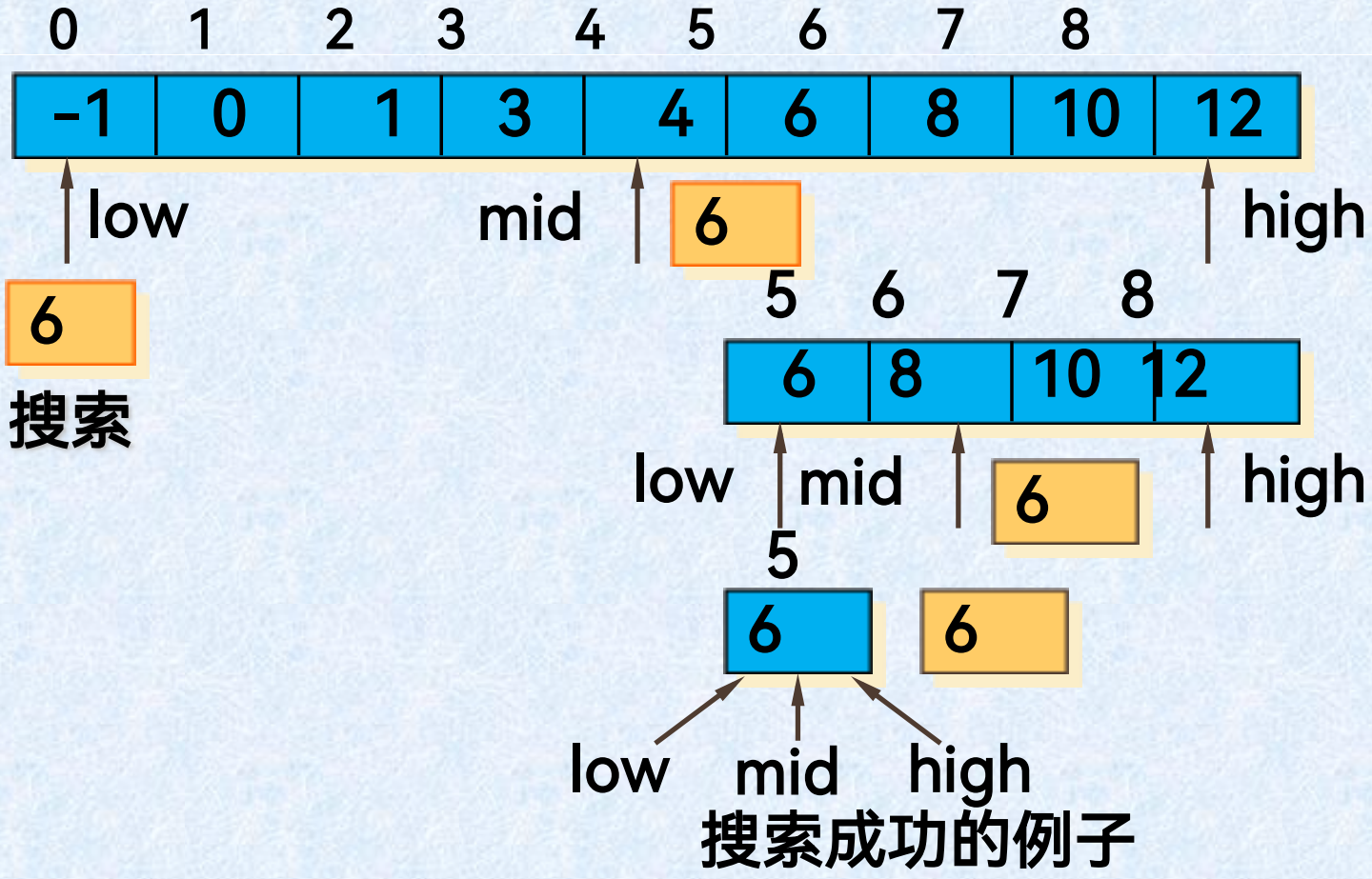
【算法分析】对于solve(*a*，0，*n*-1，max1，max2)调用，其比较次数的递推式为：

$T(1)=T(2)=1$

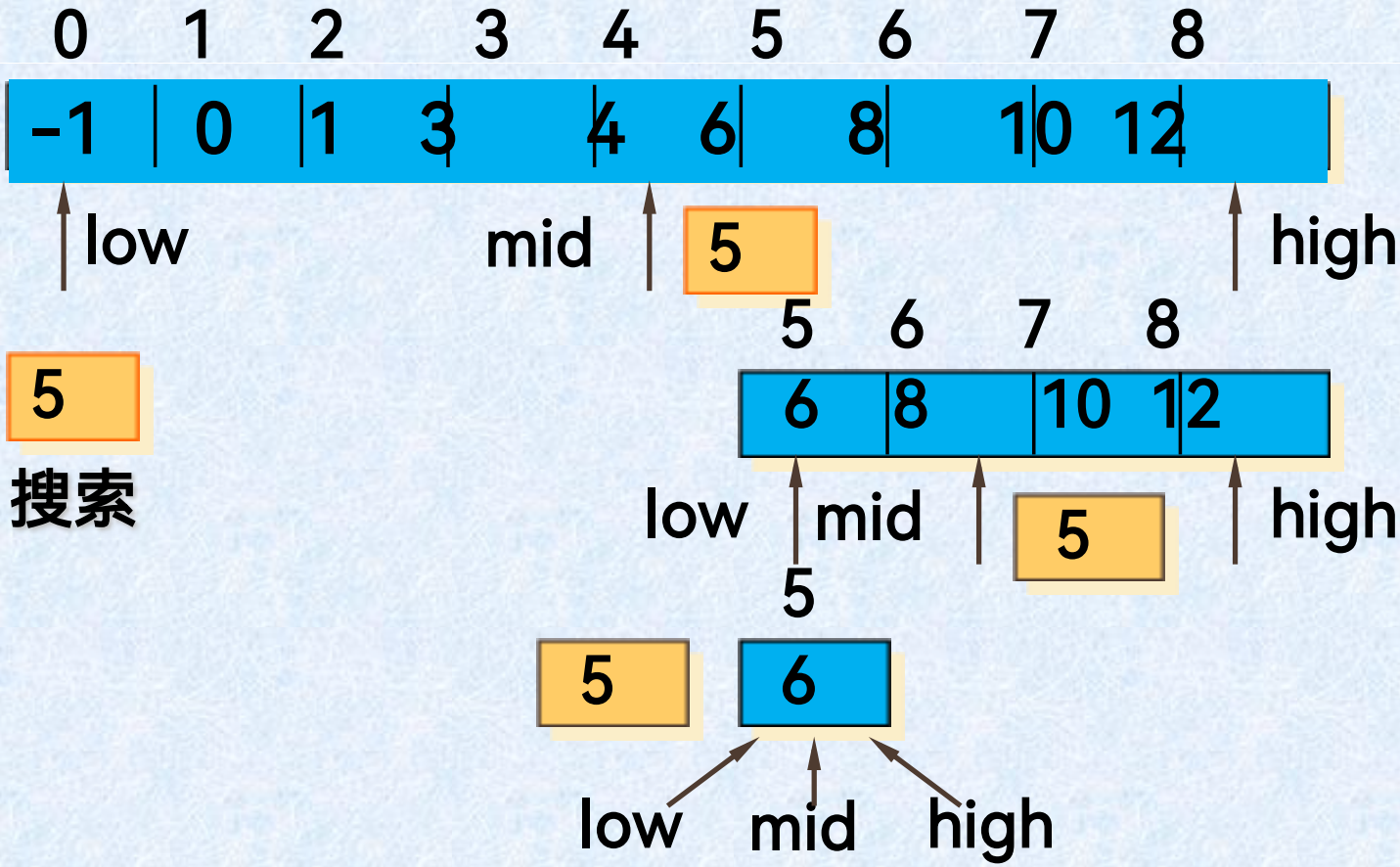
$T(n)=2T(n/2)+1$　//合并的时间为O(1)

可以推导出$T(n)=O(n)$。

2.9 折半查找



搜索成功的例子


0 1 2 3 4 5 6 7 8
-1 0 1 3 4 6 8 10 12
low
mid
5
high
5
搜索
5 6 7 8
6 8 10 12
low
mid
5
high
5
5
6
low
mid
high

搜索失败的例子

基本思路：设*a*[low..high]是当前的查找区间，首先确定该区间的中点位置mid=⌊(low+high)/2⌋；然后将待查的*k*值与*a*[mid].key比较：

（1）若*k*==*a*[mid]，则查找成功并返回该元素的物理下标；

（2）若*k*<*a*[mid]，则由表的有序性可知*a*[mid..high]均大于*k*，因此若表中存在关键字等于*k*的元素，则该元素必定位于左子表*a*[low..mid-1]中，故新的查找区间是左子表*a*[low..mid-1]；

（3）若*k*>*a*[mid]，则要查找的*k*必在位于右子表*a*[mid+1..high]中，即新的查找区间是右子表*a*[mid+1..high]。

下一次查找是针对新的查找区间进行的。

```
int BinSearch(int a[ ]，int low，int high，int k)
//拆半查找算法
{  int mid;
   if (low<=high)              //当前区间存在元素时
   {        mid=(low+high)/2;        //求查找区间的中间位置
             if (a[mid]==k)              //找到后返回其物理下标mid
                   return mid;
             if (a[mid]>k)               //当a[mid]>k时
                   return BinSearch(a，low，mid-1，k);
             else                        //当a[mid]<k时
                  return BinSearch(a，mid+1，high，k);
   }
     else return -1;                     //若当前查找区间没有元素时返回-1
}
```

【算法分析】折半查找算法的主要时间花费在元素比较上，对于含有n个元素的有序表，采用折半查找时最坏情况下的元素比较次数为$C(n)$，则有：

$$C(n)=1 \quad \text{当} n=1$$

$$C(n)\leq 1+C(\lfloor n/2 \rfloor) \quad \text{当} n\geq 2$$

由此得到：$C(n)\leq \lfloor \log_2 n \rfloor+1$

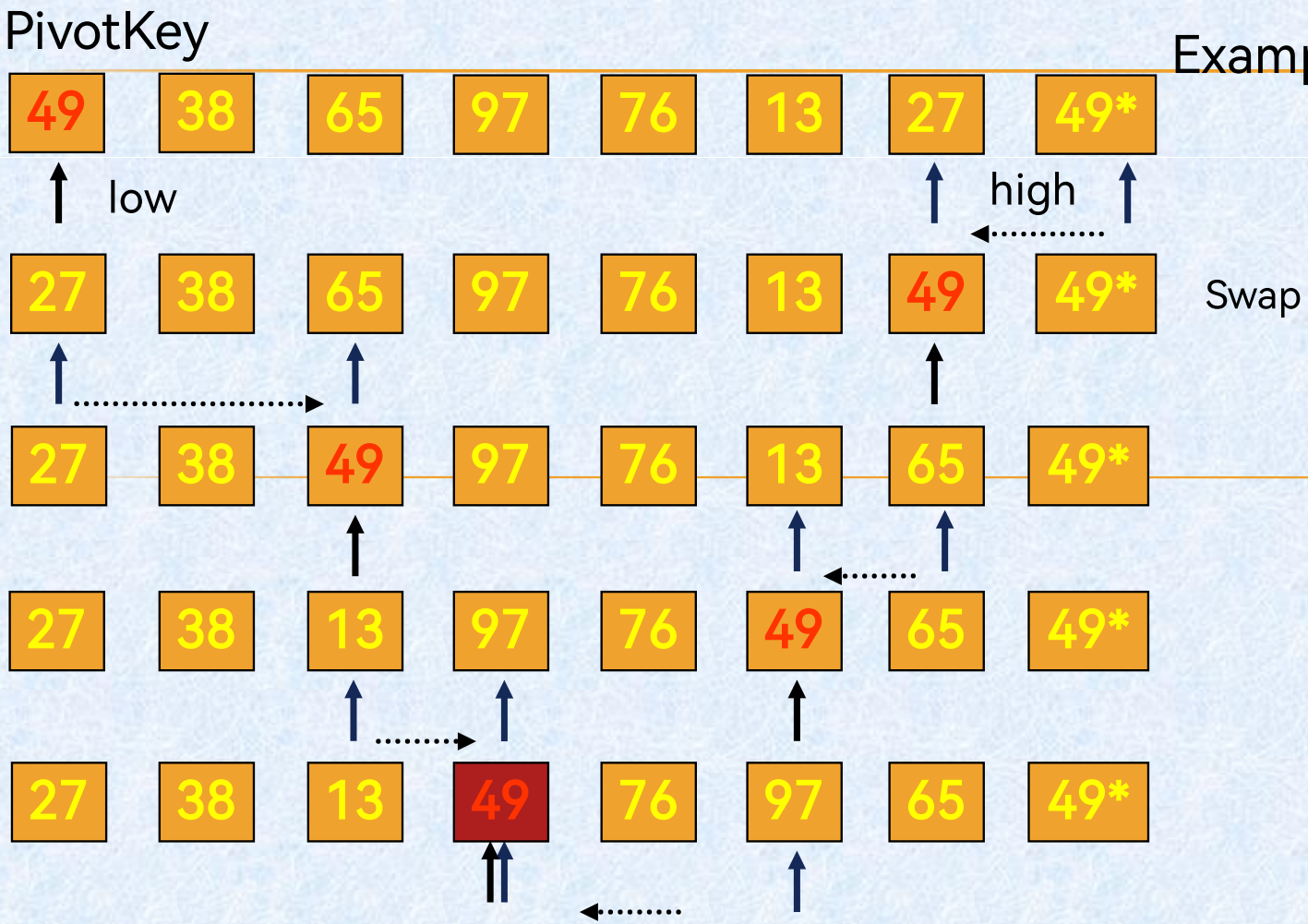
折半查找的主要时间花在元素比较上，所以算法的时间复杂度为$O(\log_2 n)$。

2.10 寻找第k小元素

【问题描述】对于给定的含有n元素的无序序列，求这个序列中第k（$1 \le k \le n$）小的元素。

【问题求解】假设无序序列存放在$a[0..n-1]$中，若将a递增排序，则第k小的元素为$a[k-1]$。

采用类似于快速排序的思想。


PivotKey
Example
49 38 65 97 76 13 27 49*
low
high
27 38 65 97 76 13 49 49*
Swap
27 38 49 97 76 13 65 49*
27 38 13 97 76 49 65 49*
27 38 13 49 76 97 65 49*

该元素的下标为$k-1$

↕

对于序列$a[s..t]$，在其中查找第k小元素的过程如下：

将$a[s]$作为基准划分，其对应下标为i。3种情况：

- 若$k-1=i$，$a[i]$即为所求，返回$a[i]$。
- 若$k-1<i$，第k小的元素应在$a[s..i-1]$子序列中，递归在该子序列中求解并返回其结果。
- 若$k-1>i$，第k小的元素应在$a[i+1..t]$子序列中，递归在该子序列中求解并返回其结果。

```
int QuickSelect(int a[ ], int s, int t, int k)
//在a[s..t]序列中找第k小的元素
{  int i=s, j=t,tmp;
   if (s<t)
   {  tmp=a[s];
      while (i!=j)                    //从区间两端交替向中间扫描，直至i=j为止
      {  while (j>i && a[j]>=tmp) j--;
                  a[i]=a[j];          //将a[j]前移到a[i]的位置
                  while (i<j && a[i]<=tmp) i++;
                  a[j]=a[i];          //将a[i]后移到a[j]的位置
      }
```

```
    a[i]=tmp;
     if (k-1==i) return a[i];
     else if (k-1<i) return QuickSelect(a, s, i-1, k);
                                        //在左区间中递归查找
     else return QuickSelect(a, i+1, t, k);
                                        //在右区间中递归查找
  }
  else if (s==t && s==k-1)     //区间内只有一个元素且为a[k-1]
     return a[k-1];
}
```

【算法分析】对于QuickSelect(*a*，*s*，*t*，*k*)算法，设序列*a* 中含有*n*个元素，其比较次数的递推式为：

$T(n)=T(n/2)+O(n)$

可以推导出$T(n)=O(n)$，这是最好的情况，即每次划分的基准恰好是中位数，将一个序列划分为长度大致相等的两个子序列。

在最坏情况下，每次划分的基准恰好是序列中的最大值或最小值，则处理区间只比上一次减少1个元素，此时比较次数为$O(n^2)$。

在平均情况下该算法的时间复杂度为$O(n)$。

2.11 寻找两个有序序列的中位数

【问题描述】对于一个长度为n的有序序列（假设均为升序序列）$a[0..n-1]$，处于中间位置的元素称为a的中位数。

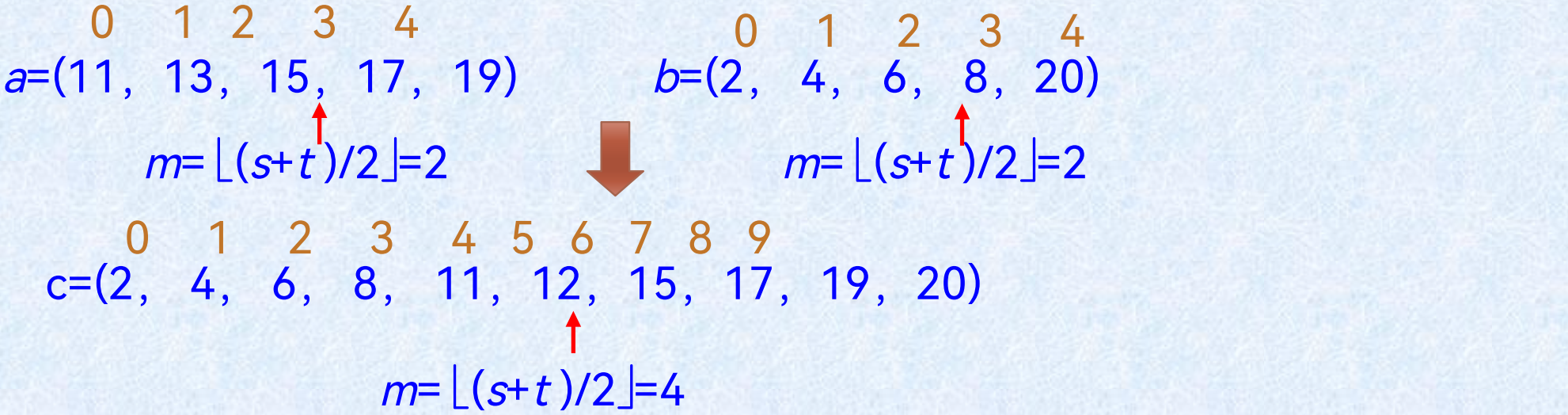
设计一个算法求给定的两个有序序列的中位数。

例如，若序列*a*=(11，13，15，17，19)，其中位数是15，若*b*=(2，4，6，8，20)，其中位数为6。两个等长有序序列的中位数是含它们所有元素的有序序列的中位数，例如*a*、*b*两个有序序列的中位数为11。

a=(11，13，15，17，19)　*b*=(2，4，6，8，20)

c=(2，4，6，8，11，13，15，17，19，20)

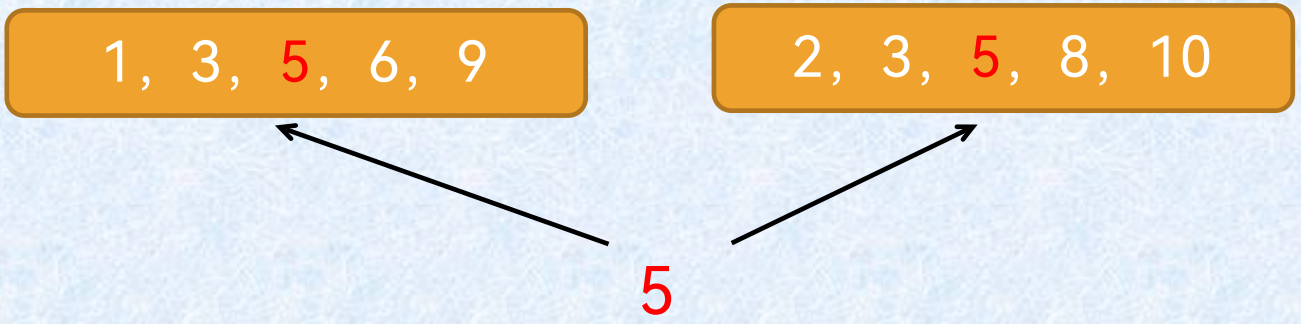
【问题求解】对于含有n个元素的有序序列$a[s..t]$，当n为奇数时，中位数是出现在$m=\lfloor(s+t)/2\rfloor$处；当n为偶数时，中位数下标有$m=\lfloor(s+t)/2\rfloor$（下中位）和$m=\lfloor(s+t)/2\rfloor+1$（上中位）两个。为了简单，仅考虑中位数为$m=\lfloor(s+t)/2\rfloor$处。



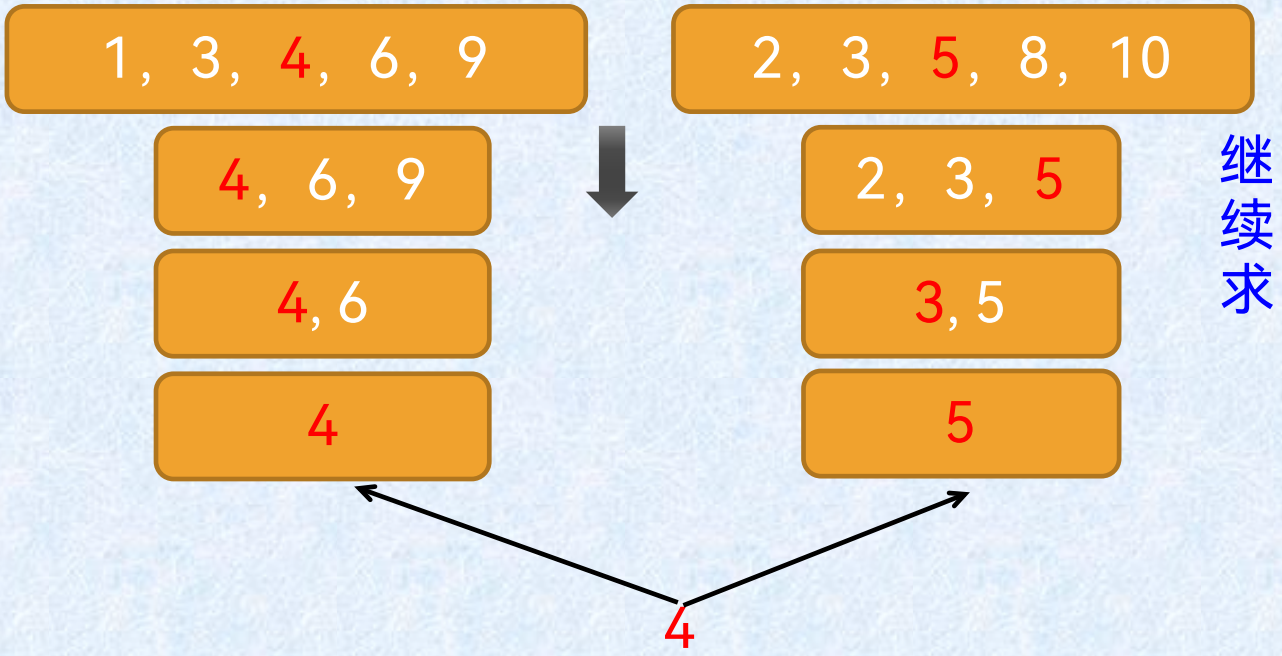
采用二分法求含有*n* 个有序元素的序列*a*、*b*的中位数的过程如下:

分别求出*a*、*b* 的中位数*a*[m1]和*b*[m2]:

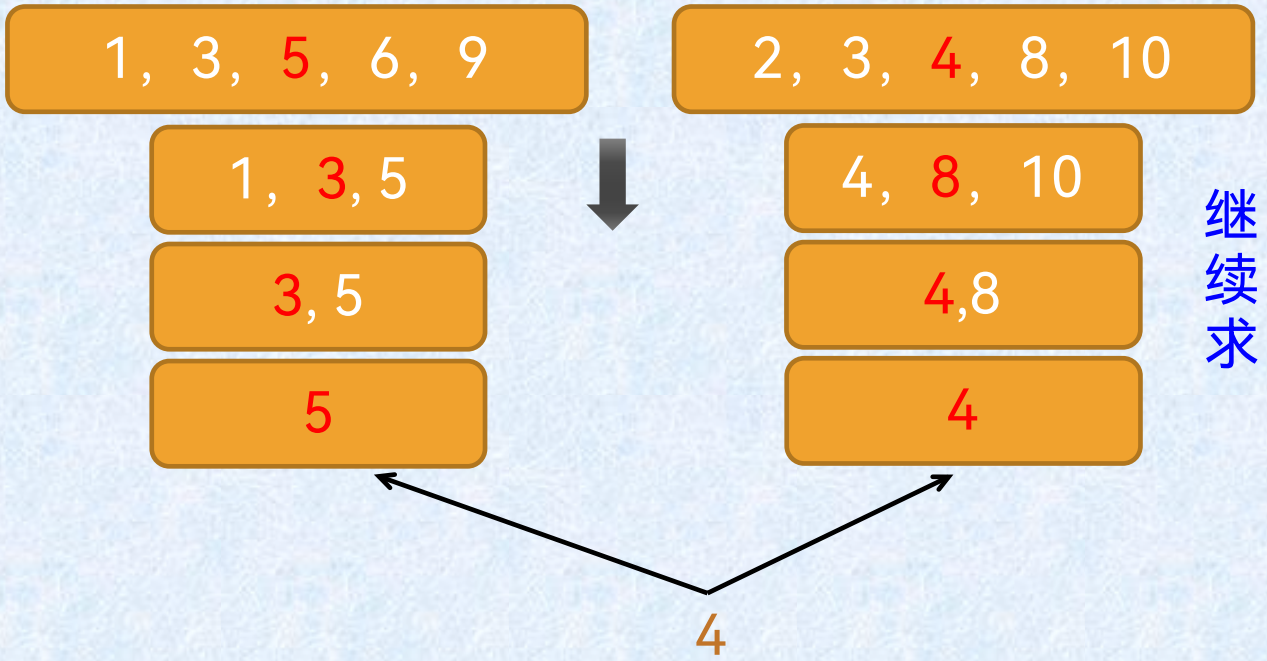
① 若*a*[m1]=*b*[m2]，则*a*[m1]或*b*[m2]即为所求中位数，算法结束。



② 若*a*[m1]<*b*[m2]，则舍弃序列*a*中前半部分（较小的一半），同时舍弃序列*b*中后半部分（较大的一半）要求舍弃的长度相等。



③ 若*a*[m1]>*b*[m2]，则舍弃序列*a*中后半部分（较大的一半），同时舍弃序列*b*中前半部分（较小的一半），要求舍弃的长度相等。



```
void prepart (int &s, int&t)
{
  int m=(s+t)/2;
   t=m;
}

Void postpart(int &s, int&t)
{
   int m=(s+t)/2;
   if((s+t)%2==0) //序列中有奇数个元素
        s=m;
   else s=m+1;//序列中有偶数个元素
}
```

```
int midnum(int a[ ], int s1, int t1, int b[ ], int s2, int t2)
{  //求两个有序序列a[s1..t1]和b[s2..t2]的中位数
      int m1, m2;
      if (s1==t1 && s2==t2)     //两序列只有一个元素时返回较小
者
            return a[s1]<b[s2]?a[s1]:b[s2];

      else
      {  m1=(s1+t1)/2;          //求a的中位数
         m2=(s2+t2)/2;          //求b的中位数
         if (a[m1]==b[m2])      //两中位数相等时返回该中位数
                  return a[m1];
```

```
        if (a[m1]<b[m2])          //当a[m1]<b[m2]时
         {  postpart(s1，t1);     //a取后半部分
             prepart(s2，t2);     //b取前半部分
               return midnum(a，s1，t1，b，s2，t2);
         }
        else                      //当a[m1]>b[m2]时
        { prepart(s1，t1);        //a取前半部分
           postpart(s2，t2);      //b取后半部分
           return midnum(a，s1，t1，b，s2，t2);
         }
      }
}
```

【算法分析】对于含有n个元素的有序序列a和b，设调用midnum(a，0，n-1，b，0，n-1)求中位数的执行时间为$T(n)$，显然有以下递归式：

$$T(n)=1 \qquad \text{当} n=1$$

$$T(n)=T(n/2)+1 \qquad \text{当} n>1$$

容易推出，$T(n)=O(\log_2 n)$。

2.12 求最大连续子序列和

【问题描述】给定一个有n（n≥1）个整数的序列，要求求出其中最大连续子序列的和。

例如:

序列（-2，11，-4，13，-5，-2）的最大子序列和为20

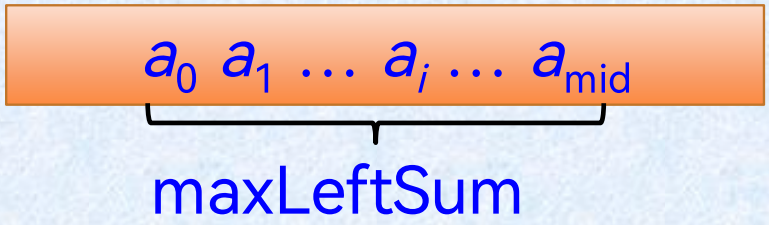
序列（-6，2，4，-7，5，3，2，-1，6，-9，10，-2）的最大子序列和为16。

规定一个序列最大连续子序列和至少是0，如果小于0，其结果为0。

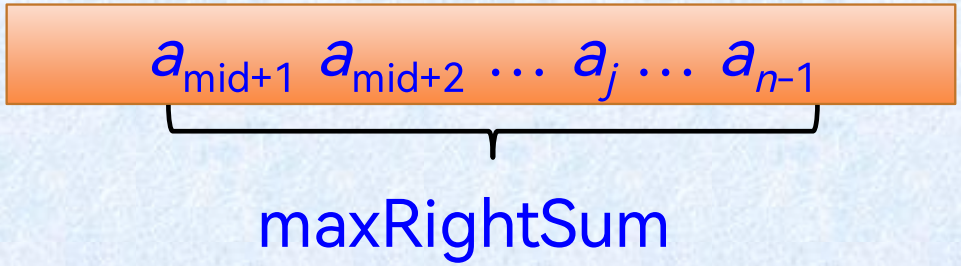
【问题求解】对于含有n个整数的序列$a[0..n-1]$，若$n=1$，表示该序列仅含一个元素，如果该元素大于0，则返回该元素；否则返回0。

若$n>1$，采用分治法求解最大连续子序列时，取其中间位置$mid=\lfloor (n-1)/2 \rfloor$，该子序列只可能出现3个地方。

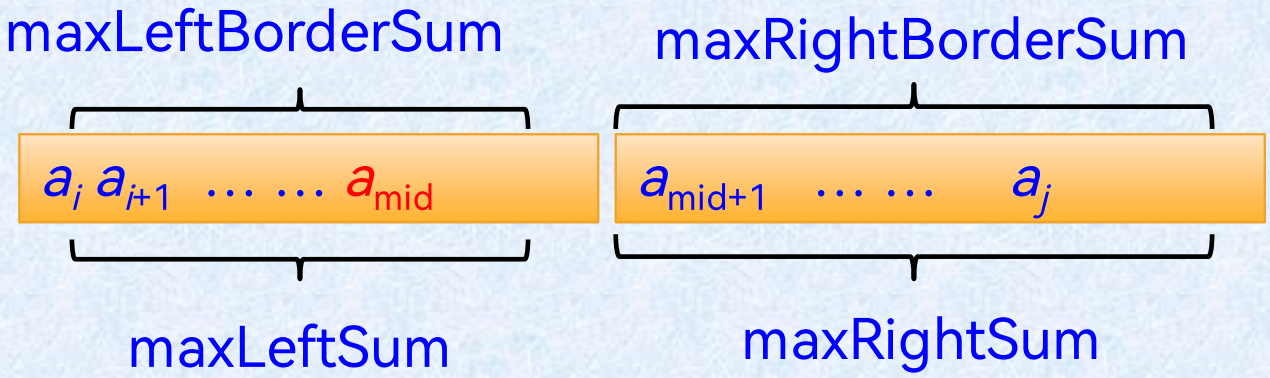
（1）该子序列完全落在左半部即$a[0..mid]$中。采用递归求出其最大连续子序列和maxLeftSum。



（2）该子序列完全落在右半部即a[mid+1..n-1]中。采用递归求出其最大连续子序列和maxRightSum。



（3）该子序列跨越序列*a*的中部而占据左右两部分。



结果： max3(maxLeftSum,
maxRightSum,
maxLeftBorderSum+maxRightBorderSum)

```
long maxSubSum(int a[ ]，int left，int right)
//求a[left..high]序列中最大连续子序列和
{  int i，j;
   long maxLeftSum，maxRightSum;
   long maxLeftBorderSum，leftBorderSum;
   long maxRightBorderSum，rightBorderSum;

   if (left==right)                    //子序列只有一个元素时
   {      if (a[left]>0)    //该元素大于0时返回它
                  return a[left];
            else                       //该元素小于或等于0时返回0
                     return 0;
   }
```

```
    maxRightBorderSum=0，rightBorderSum=0;
     for (j=mid+1;j<=right;j++)                    //求出a[mid]右边
元素
     {  rightBorderSum+=a[j];                      //构成的序列的最
大和
        if (rightBorderSum>maxRightBorderSum)
                maxRightBorderSum=rightBorderSum;
     }
     return max3(maxLeftSum，maxRightSum，
                 maxLeftBorderSum+maxRightBorderSum);
}
```

```
    maxRightBorderSum=0，rightBorderSum=0;
     for (j=mid+1;j<=right;j++)                    //求出a[mid]右
边元素
     {  rightBorderSum+=a[j];                      //构成的序列的
最大和
        if (rightBorderSum>maxRightBorderSum)
                maxRightBorderSum=rightBorderSum;
     }
     return max3(maxLeftSum，maxRightSum，
                 maxLeftBorderSum+maxRightBorderSum);
}
```

【算法分析】设求解序列$a[0..n-1]$最大连续子序列和的执行时间为$T(n)$，第（1）、（2）两种情况的执行时间为$T(n/2)$，第（3）种情况的执行时间为$O(n)$，所以得到以下递推式：

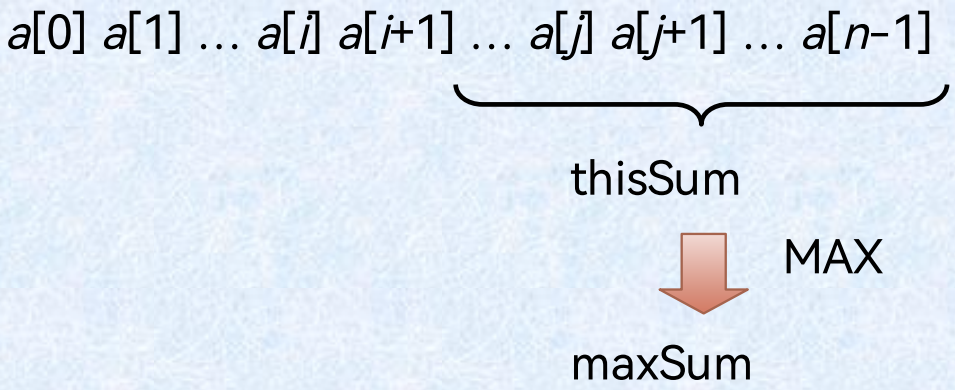
$T(n)=1$ 当$n=1$

$T(n)=2T(n/2)+n$ 当$n>1$

容易推出，$T(n)=O(n\log_2 n)$。

解法1：设含有n个整数的序列$a[0..n-1]$，其中任何连续子序列$a[i..j]$（$i \le j$，$0 \le i \le n-1$，$i \le j \le n-1$）求出它的所有元素之和thisSum。

通过比较将最大值存放在maxSum中，最后返回maxSum。



i：0 ~ $n-1$

j：i ~ $n-1$

} k：i ~ j

```
int maxSubSum1(int a[],int n)
{  int i,j,k;
   int maxSum=a[0],thisSum;
   for (i=0;i<n;i++) //两重循环穷举所有的连续子序列
   {  for (j=i;j<n;j++)
      {  thisSum=0;
         for (k=i;k<=j;k++)
                  thisSum+=a[k];
            if (thisSum>maxSum) //通过比较求最大连续子序列之和
                  maxSum=thisSum;
         }
    }
    return maxSum;
}
```

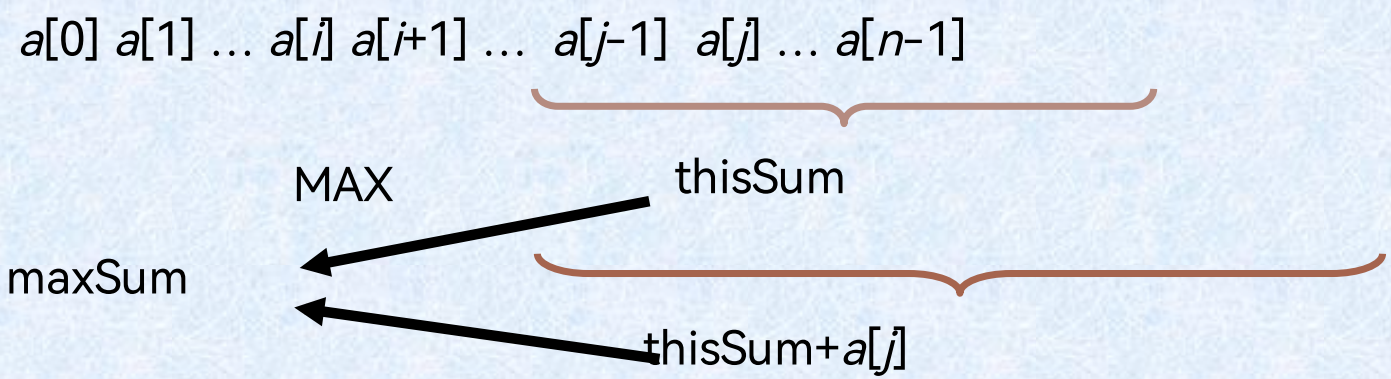
maxSubSum1(*a*,*n*)算法中用了三重循环，所以有：

$$T(n)=\sum_{i=0}^{n-1}\sum_{j=i}^{n-1}\sum_{k=i}^{j}1=\sum_{i=0}^{n-1}\sum_{j=i}^{n-1}(j-i+1)=\frac{1}{2}\sum_{i=0}^{n-1}(n-i)(n-i+1)$$

$O(n^3)$

解法2：改进前面的解法，在求两个相邻子序列和时，它们之间是关联的。

例如*a*[0..3]子序列和=*a*[0]+*a*[1]+*a*[2]+*a*[3]，*a*[0..4]子序列和=*a*[0]+*a*[1]+*a*[2]+*a*[3]+*a*[4]，在前者计算出来后，求后者时只需在前者基础上加以*a*[4]即可，没有必须每次都重复计算。从而提高了算法效率。



i: 0 ~ *n*-1

j: *i* ~ *n*-1

```
int maxSubSum2(int a[],int n)
{  int i,j;
   int maxSum=a[0],thisSum;
   for (i=0;i<n;i++)
   {  thisSum=0;
      for (j=i;j<n;j++)
      {  thisSum+=a[j]; //maxSum已经包含了a[i..j-1]的最大和
                if (thisSum>maxSum)
                    maxSum=thisSum;
      }
   }
   return maxSum;
}
```

maxSubSum2(*a*,*n*)算法中只有两重循环，容易求出T(*n*)=O(n^2)。

解法3：更一步改进解法2。

如果扫描中遇到负数，当前子序列和thisSum将会减小，若thisSum为负数，表明前面已经扫描的那个子序列可以抛弃了，则放弃这个子序列，重新开始下一个子序列的分析，并置thisSum为0。

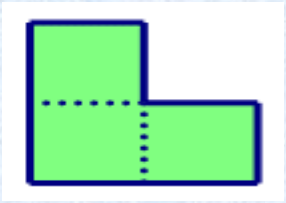
若这个子序列和thisSum不断增加，那么最大子序列和maxSum也不断增加。

```
int maxSubSum3(int a[],int n)
{  int i,maxSum=0,thisSum=0;
   for (i=0;i<n;i++)
   {  thisSum+=a[i];
      if (thisSum<0) //若当前子序列和为负数，重新开始下一子序列
                thisSum=0;
      if (maxSum<thisSum)   //比较求最大连续子序列和
                maxSum=thisSum;
   }
   return maxSum;
}
```

显然该算法中仅扫描*a*一次，其算法的时间复杂度为O(*n*)。

2.13 求解棋盘覆盖问题

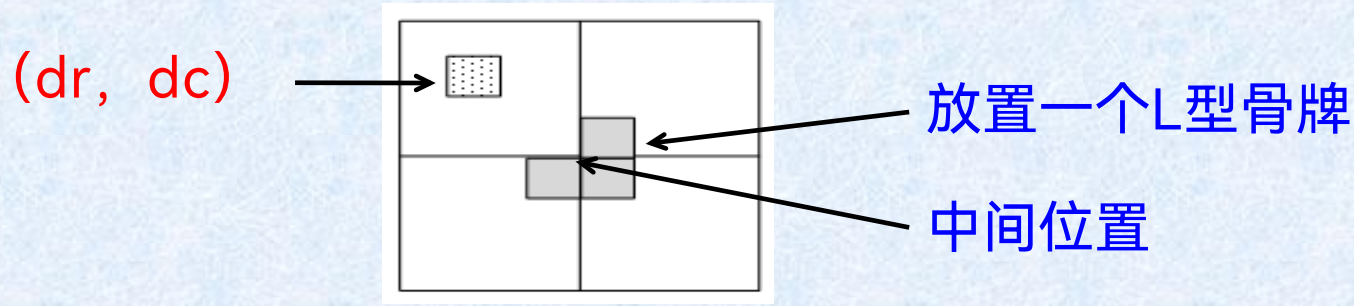
【问题描述】有一个$2^k \times 2^k$（$k>0$）的棋盘，恰好有一个方格与其他方格不同，称之为特殊方格。现在要用如下的L型骨牌覆盖除了特殊方格外的其他全部方格，骨牌可以任意旋转，并且任何两个骨牌不能重叠。请给出一种覆盖方法。



【问题求解】棋盘中的方格数=$2^k \times 2^k = 4^k$，覆盖使用的L型骨牌个数=$(4^k-1)/3$。

采用的方法是：将棋盘划分为4个大小相同4个象限，根据特殊方格的位置（dr，dc），在中间位置放置一个合适的L型骨牌。

例如，如下图所示，特殊方格在左上角象限中，在中间放置一个覆盖其他3个象限中各一个方格的L型骨牌。



特殊方格在左上角象限

其他情况类似!

用（tr，tc）表示一个象限左上角方格的坐标，（dr，dc）是特殊方格所在的坐标，size是棋盘的行数和列数。

用二维数组board存放覆盖方案，用tile全局变量表示L型骨牌的编号（从整数1开始），board中3个相同的整数表示一个L型骨牌。

```
#include<stdio.h>
#define MAX 1025
//问题表示
int k;      //棋盘大小
int x,y;    //特殊方格的位置

//求解问题表示
int board[MAX][MAX];
int tile=1;
```

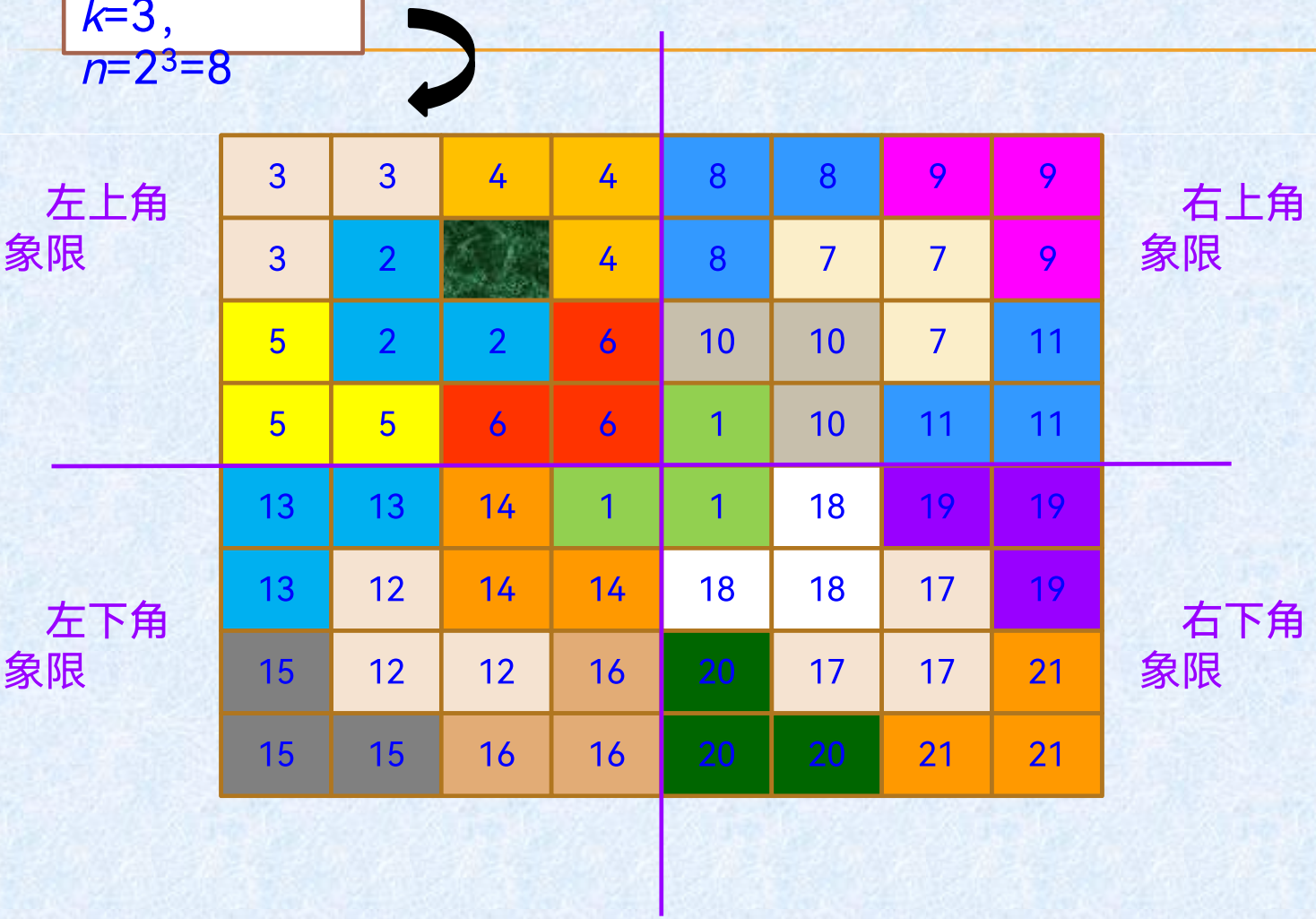
```
void ChessBoard(int tr,int tc,int dr,int dc,int size)
{   if(size==1) return;                          //递归出口
    int t=tile++;                                //取一个L型骨，其牌号为tile
    int s=size/2;                                //分割棋盘
    //考虑左上角象限
    if(dr<tr+s && dc<tc+s)                       //特殊方格在此象限中
            ChessBoard(tr,tc,dr,dc,s);
    else                                         //此象限中无特殊方格
    {       board[tr+s-1][tc+s-1]=t;             //用t号L型骨牌覆盖右下角
            ChessBoard(tr,tc,tr+s-1,tc+s-1,s);
                                                 //将右下角作为特殊方格继续处理该象限
    }
    //考虑右上角象限
    if(dr<tr+s && dc>=tc+s)
            ChessBoard(tr,tc+s,dr,dc,s);                    //特殊方格在此象限中
    else                                                    //此象限中无特殊方格
    {       board[tr+s-1][tc+s]=t;                          //用t号L型骨牌覆盖左下角
            ChessBoard(tr,tc+s,tr+s-1,tc+s,s);
                                                 //将左下角作为特殊方格继续处理该象限
    }
```

```
    //处理左下角象限
    if(dr>=tr+s && dc<tc+s)              //特殊方格在此象限中
            ChessBoard(tr+s,tc,dr,dc,s);
    else                                 //此象限中无特殊方格
    {   board[tr+s][tc+s-1]=t;           //用t号L型骨牌覆盖右上角
        ChessBoard(tr+s,tc,tr+s,tc+s-1,s);
                                         //将右上角作为特殊方格继续处理该象限
    }
    //处理右下角象限
    if(dr>=tr+s && dc>=tc+s)                    //特殊方格在此象限中
        ChessBoard(tr+s,tc+s,dr,dc,s);
    else                                 //此象限中无特殊方格
    {       board[tr+s][tc+s]=t;         //用t号L型骨牌覆盖左上角
            ChessBoard(tr+s,tc+s,tr+s,tc+s,s);
                                         //将左上角作为特殊方格继续处理该象限
    }
}
```

$k=3$,
$n=2^3=8$

3	3	4	4	8	8	9	9
3	2	0	4	8	7	7	9
5	2	2	6	10	10	7	11
5	5	6	6	1	10	11	11
13	13	14	1	1	18	19	19
13	12	14	14	18	18	17	19
15	12	12	16	20	17	17	21
15	15	16	16	20	20	21	21

$k=3$,
$n=2^3=8$


左上角
象限
右上角
象限
左下角
象限
右下角
象限
3 3 4 4 8 8 9 9
3 2 4 8 7 7 9
5 2 2 6 10 10 7 11
5 5 6 6 1 10 11 11
13 13 14 1 1 18 19 19
13 12 14 14 18 18 17 19
15 12 12 16 20 17 17 21
15 15 16 16 20 20 21 21

谢 谢